

DDS Compiler v6.0

LogiCORE IP Product Guide

Vivado Design Suite

PG141 (v6.0) May 30, 2024

AMD Adaptive Computing is creating an environment where employees, customers, and partners feel welcome and included. To that end, we're removing non-inclusive language from our products and related collateral. We've launched an internal initiative to remove language that could exclude people or reinforce historical biases, including terms embedded in our software and IPs. You may still find examples of non-inclusive language in our older products as we work to make these changes and align with evolving industry standards. Follow this [link](#) for more information.



Table of Contents

Chapter 1: Introduction.....	4
Features.....	4
IP Facts.....	5
Chapter 2: Overview.....	6
Navigating Content by Design Process.....	6
Core Overview.....	6
Applications.....	7
Licensing and Ordering.....	7
Chapter 3: Product Specification.....	8
Phase Generator.....	9
SIN/COS LUT.....	9
Phase Generator and SIN/COS LUT (DDS).....	10
Performance.....	10
Resource Use.....	10
Port Descriptions.....	11
Chapter 4: Designing with the Core.....	14
General Design Guidelines.....	14
General Description.....	15
Theory of Operation.....	15
Multichannel.....	32
Design Examples.....	33
Clocking.....	33
Resets.....	34
Protocol Description.....	35
Chapter 5: Design Flow Steps.....	44
Customizing and Generating the Core.....	44
System Generator for DSP.....	55
Constraining the Core/Subsystem.....	55



Simulation.....	56
Synthesis and Implementation.....	56
Chapter 6: C Model.....	57
Features.....	57
Overview.....	57
Unpacking and Model Contents.....	58
Installation.....	58
C Model Interface.....	59
Data Format.....	63
Compiling.....	63
Linking.....	63
Example.....	64
MATLAB Interface.....	65
Chapter 7: Test Bench.....	68
Demonstration Test Bench.....	68
Appendix A: Upgrading.....	71
Migrating to the Vivado Design Suite.....	71
Upgrading in the Vivado Design Suite.....	71
Appendix B: Debugging.....	77
Finding Help with AMD Adaptive Computing Solutions.....	77
Debug Tools.....	79
Interface Debug.....	79
Appendix C: Additional Resources and Legal Notices.....	81
Finding Additional Documentation.....	81
Support Resources.....	82
References.....	82
Revision History.....	82
Please Read: Important Legal Notices.....	83

Introduction

The AMD LogiCORE™ IP Direct Digital Synthesizer (DDS) Compiler core implements high performance, optimized phase generation and phase to sinusoid circuits with AXI4-Stream compliant interfaces.

The core sources sinusoidal waveforms for use in many applications. A DDS consists of a Phase Generator and a SIN/COS Lookup Table (phase to sinusoid conversion). These parts are available individually or combined using this core.

Features

- The Phase Generator and SIN/COS LUT can be generated individually or together with optional dither to provide a complete DDS solution.
- Rasterized feature eliminates phase noise from phase truncation.
- Sine, cosine, or quadrature outputs
- Integer or float sine/cosine outputs
- Optional per-channel resynchronization of accumulated phase
- Lookup table can be stored in distributed or block RAM
- Optional phase dithering spreads the spectral line energy for greater spurious-free dynamic range (SFDR)
- Phase dithering or Taylor series correction options provide high dynamic range signals using minimal FPGA resources. Supports SFDR from 18 dB to 150 dB.
- Up to 16 independent time-multiplexed channels
- Fine frequency resolution using up to 48-bit phase accumulator with DSP slice or FPGA logic options
- 3-bit to 26-bit signed (integer) or 32-bit (single precision float) output sample width

IP Facts

AMD LogiCORE™ IP Facts Table	
Core Specifics	
Supported Device Family ¹	AMD Versal™ adaptive SoC, AMD UltraScale+™ devices, AMD UltraScale™ devices, AMD Zynq™ 7000 SoC devices, 7 series devices
Supported User Interfaces	AXI4-Stream
Resources	Performance and Resource Use web page
Provided with Core	
Design Files	Encrypted RTL
Example Design	Not Provided
Test Bench	VHDL
Constraints File	Not Provided
Simulation Model	Encrypted VHDL
Supported S/W Driver	N/A
Tested Design Flows ²	
Design Entry	AMD Vivado™ Design Suite System Generator for DSP
Simulation	For supported simulators, see the <i>Vivado Design Suite User Guide: Release Notes, Installation, and Licensing</i> (UG973).
Release Notes and Known Issues	Master Answer Record: 54498
All Vivado IP Change Logs	Master Vivado IP Change Logs: 72775
Support web page	

Notes:

1. For a complete list of supported devices, see the Vivado IP catalog.
2. For the supported versions of third-party tools, see the *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* ([UG973](#)).

Overview

Navigating Content by Design Process

AMD Adaptive Computing documentation is organized around a set of standard design processes to help you find relevant content for your current development task. You can access the AMD Versal™ adaptive SoC design processes on the [Design Hubs](#) page. You can also use the [Design Flow Assistant](#) to better understand the design flows and find content that is specific to your intended design needs.

- **Hardware, IP, and Platform Development:** Creating the PL IP blocks for the hardware platform, creating PL kernels, functional simulation, and evaluating the AMD Vivado™ timing, resource use, and power closure. Also involves developing the hardware platform for system integration. Topics in this document that apply to this design process include:
 - [Port Descriptions](#)
 - [Clocking](#)
 - [Resets](#)
 - [Customizing and Generating the Core](#)
-

Core Overview

Direct digital synthesizers (DDS), or numerically controlled oscillators (NCO), are important components in many digital communication systems. Quadrature synthesizers are used for constructing digital down and up converters and demodulators. They are also used for implementing various types of modulation schemes, including PSK (phase shift keying), FSK (frequency shift keying), and MSK (minimum shift keying). A common method for digitally generating a complex or real valued sinusoid employs a lookup table scheme. The lookup table stores samples of a sinusoid. A digital integrator is used to generate a suitable phase argument that is mapped by the lookup table to the desired output waveform. A simple user interface accepts system-level parameters such as the desired output frequency and spur suppression of the generated waveforms.

Applications

The DDS Compiler core is used in applications such as:

- Digital radios and modems
- Software-defined radios (SDR)
- Digital down/up converters for cellular and PCS base stations
- Waveform synthesis in digital phase locked loops
- Generating injection frequencies for analog mixers

Licensing and Ordering

This AMD LogiCORE™ IP module is provided at no additional cost with the AMD Vivado™ Design Suite under the terms of the [End User License](#).

For more information about this core, visit the DDS Compiler [product web page](#).

Information about other AMD LogiCORE™ IP modules is available at the [Intellectual Property](#) page. For information about pricing and availability of other AMD LogiCORE IP modules and tools, contact your [local sales representative](#).

License Checkers

If the IP requires a license key, the key must be verified. The AMD Vivado™ design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation
- write_bitstream (Tcl command)



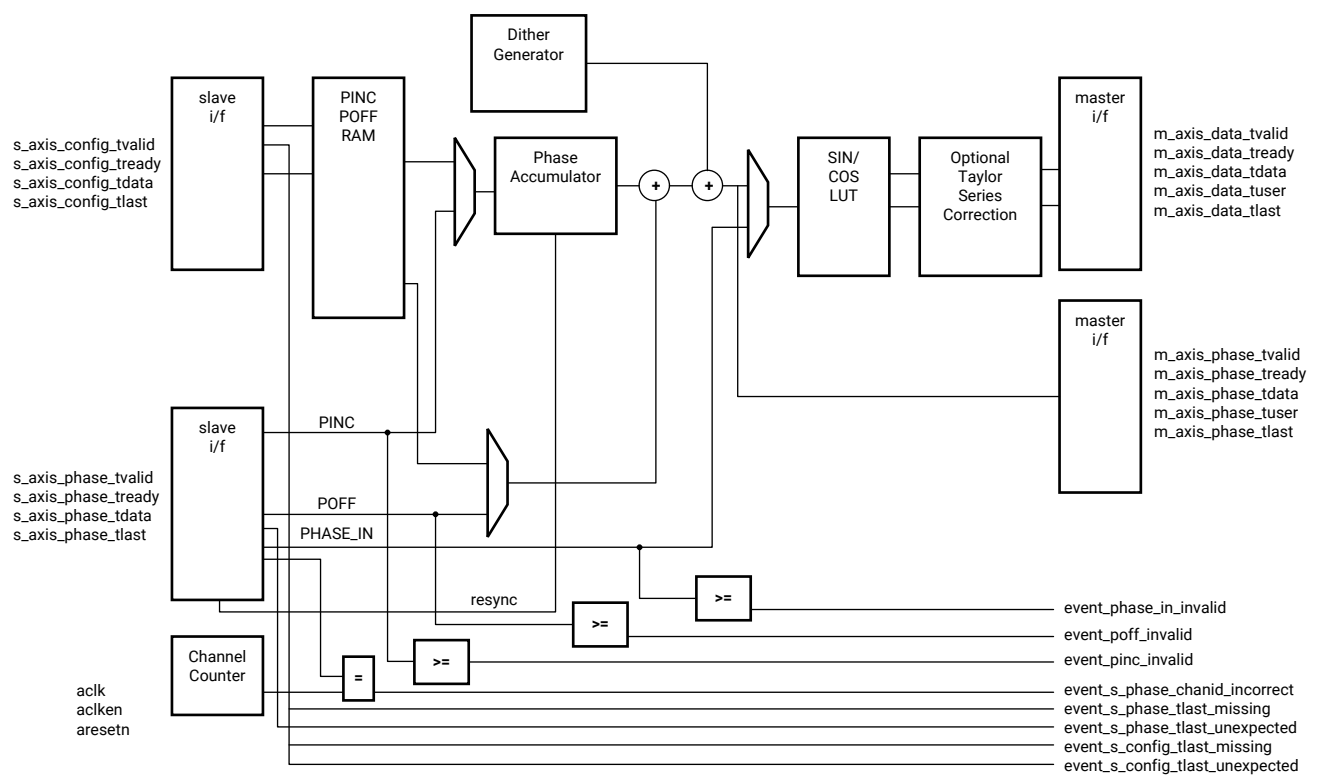
IMPORTANT! IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.

Product Specification

The functional block diagram of the core is shown in the following figure.

The core consists of two main parts, a Phase Generator and a SIN/COS LUT, which can be used independently or together with an optional dither generator to create a DDS capability. A time-division (TDM) multichannel capability is supported, with independently configurable phase increment and offset parameters.

Figure 1: DDS Compiler Core Architecture



X13213

Phase Generator

The Phase Generator consists of an accumulator followed by an optional adder to allow the addition of a phase offset. When the core is customized, the phase increment (PINC) and phase offset (POFF) can be independently configured to be either fixed, programmable (using the CONFIG channel), or streaming (using the input PHASE channel).

When set to fixed, the DDS output frequency is set when the core is customized and cannot be adjusted after the core is embedded in a design.

When set to programmable, the CONFIG channel TDATA field has a subfield for the input in question (PINC or POFF) or both if both have been selected to be programmable. If neither PINC nor POFF is set to programmable, there is no CONFIG channel.

When set to streaming, the input PHASE channel TDATA field has a subfield for the input in question (PINC or POFF) or both if both have been selected to be streaming. If neither PINC nor POFF is set to streaming, and the core is configured to have a Phase Generator, there is no input PHASE channel.

When PINC is set to streaming, an optional RESYNC streaming input can be configured. When asserted, this signal resets the accumulated phase of the channel in question.

When rasterized mode is selected, the hardware values of PINC and POFF that are input or configured must be 0 to Modulus-1. This corresponds to a full circle. For negative PINC or POFF values, add the Modulus to the negative value desired to map to the required range. For example, with Modulus = 100, the required range is 0 to 99. An angle of -90° would be -25 with this Modulus. Adding 100 gives 75 (270°).

When using system parameters, PINC and POFF are not input directly, but are calculated from the input output frequencies and phase angles. Be aware that for small values of Modulus, the available values are relatively far apart, so the actual output frequency or phase angle might differ significantly from the desired value as displayed in the Additional Summary tab.

SIN/COS LUT

When configured as a SIN/COS LUT only, the Phase Generator is not implemented and the `PHASE_IN` signal is input using the input PHASE channel and transformed into sine and cosine outputs using a lookup table. Efficient memory usage is achieved by exploiting the symmetry of sinusoid waveforms. The core can be configured for sine-only output, cosine-only output, or both (quadrature) output. Each output can be configured independently to be negated. Precision can be increased using optional Taylor series correction. This exploits DSP slices on FPGA families that support them to achieve high SFDR with high speed operation.

Phase Generator and SIN/COS LUT (DDS)

The Phase Generator is used with the SIN/COS LUT to provide either a phase truncated DDS or a Taylor series corrected DDS. An optional dither generator can be added between the two blocks to provide a phase dithered DDS.

Performance

This section details the performance information for various core configurations.

Latency

The latency of the core can be specified through the user interface in the Vivado Integrated Design Environment (IDE), or it can be automatically set to the optimum value based upon the optimization goal.

For streaming inputs (`s_axis_phase_t*`) the latency specifies the minimum number of cycles between input and the associated output.

For configuration inputs, the latency is the minimum latency from the first cycle `aresetn` becomes inactive until a valid output. The latency from configuration channel inputs to output channel outputs is not deterministic because configuration inputs are synchronized to the internal channel counter phase, thus introducing a delay which is unknown external to the core.

Note: It is recommended that `CE` and `TVALID` on input are *not* both used to control data into the core, because this can lead to obscure latency. For instance, if `TREADY` is not used, data samples are pushed through the core by `TVALID` on input, and setting `TVALID` Low freezes data in transit much like clock enable. It is recommended that full AXI4-Stream protocol is used where latency is variable, but no data is lost. If deterministic latency is required, fix either `CE` or `TVALID` High and control data flow using the other.

Throughput

The DDS Compiler core supports full throughput in all configurations (one output for every cycle).

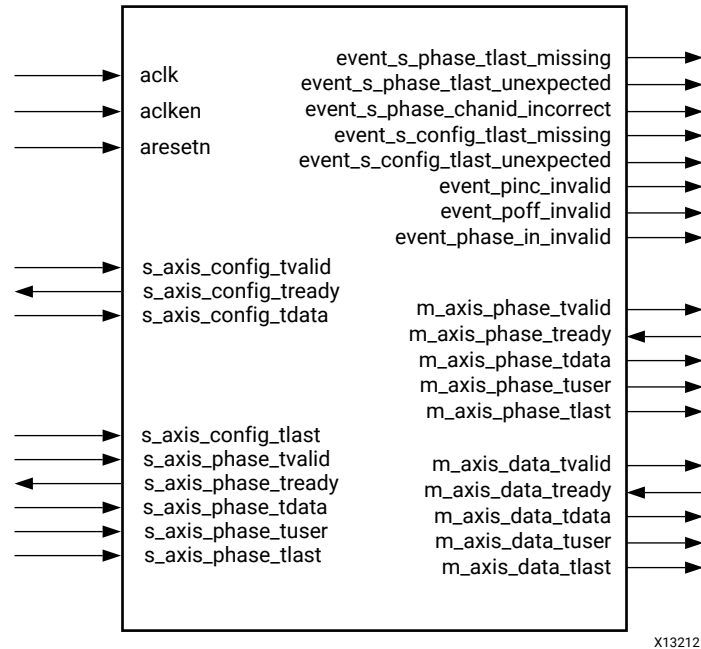
Resource Use

For full details about performance and resource use, visit the [Performance and Resource Use web page](#).

Port Descriptions

The DDS Compiler core pinout is shown in the following figure. All of the possible pins are shown, though the specific pins in any instance depend upon parameters specified when the core is generated.

Figure 2: Core Ports



X13212

The following table summarizes the pinout of the core. If an active-Low input is required for a specific control pin, an inverter must be placed in the path to the pin and is absorbed appropriately during mapping.

Table 1: Core Signal Pinout

Name	Direction	Optional	Description
aclk	I	No	Rising edge clock
aclkcn	I	Yes	Active-High clock enable
aresetn	I	Yes	Active-Low synchronous clear. Always takes priority over aclkcn. aresetn must be driven Low for a minimum of two cycles to reset the core.
s_axis_config_tvalid	I	Yes	TVALID for CONFIG channel
s_axis_config_tready	O	Yes	TREADY for CONFIG channel
s_axis_config_tdata ¹	I	Yes	TDATA for CONFIG channel. See CONFIG Channel TDATA Structure for internal structure and width.

Table 1: Core Signal Pinout (cont'd)

Name	Direction	Optional	Description
s_axis_config_tlast	I	Yes	TLAST for CONFIG channel. See CONFIG Channel .
s_axis_phase_tvalid	I	Yes	TVALID for input PHASE channel
s_axis_phase_tready	O	Yes	TREADY for input PHASE channel
s_axis_phase_tdata ¹	I	Yes	TDATA for input PHASE channel. See Input PHASE Channel TDATA Structure for internal structure and width.
s_axis_phase_tuser	I	Yes	TUSER for input PHASE channel. See Input PHASE Channel TUSER Structure for internal structure.
s_axis_phase_tlast	I	Yes	TLAST for input PHASE channel. See Input PHASE Channel TLAST Options .
m_axis_phase_tvalid	O	Yes	TVALID for output PHASE channel.
m_axis_phase_tready	I	Yes	TREADY for output PHASE channel.
m_axis_phase_tdata	O	Yes	TDATA for output PHASE channel. See Output PHASE Channel TDATA Structure – Conventional DDS .
m_axis_phase_tuser	O	Yes	TUSER for output PHASE channel. See Output PHASE Channel TUSER Structure .
m_axis_phase_tlast	O	Yes	TLAST for output PHASE channel. See Output PHASE Channel TLAST Options .
m_axis_data_tvalid	O	Yes	TVALID for output DATA channel.
m_axis_data_tready	I	Yes	TREADY for output DATA channel.
m_axis_data_tdata	O	Yes	TDATA for output DATA channel. See Output DATA Channel TDATA Structure .
m_axis_data_tuser	O	Yes	TUSER for output DATA channel. See Output DATA Channel TUSER Structure .
m_axis_data_tlast	O	Yes	TLAST for output DATA channel. See Output DATA Channel TLAST Options .

Table 1: Core Signal Pinout (cont'd)

Name	Direction	Optional	Description
event_s_phase_tlast_missing	O	No (but leave unconnected to remove associated circuitry)	Asserted when the transfer to the s_axis_phase channel for the last channel (in multichannel configurations) does not have tlast asserted.
event_s_phase_tlast_unexpected	O		Asserted when TLAST is asserted for a transfer to the s_axis_phase channel which is not for the last channel (in multichannel configurations).
event_s_phase_chanid_incorrect	O		Asserted when the chanid field (subfield of TUSER in s_axis_phase) does not agree with the internal expectation of channel number.
event_pinc_invalid	O		Asserted when the value of PINC to be accumulated is out of range. Applies to rasterized configurations only.
event_poff_invalid	O		Asserted when the value of POFF to be accumulated is out of range. Applies to rasterized configurations only.
event_phase_in_invalid	O		Asserted when the value of Phase_In is out of range. Applies to rasterized and SIN_COS_LUT configurations only.
event_s_config_tlast_missing	O		Asserted when the last transfer (for the last channel) to the s_axis_config channel is not accompanied with TLAST asserted.
event_s_config_tlast_unexpected	O		Asserted when a transfer to the s_axis_config channel has TLAST asserted when it is not the transfer associated with the last channel.

Notes:

- When in rasterized mode, the input values of PINC, POFF, or PHASE_IN must be between 0 and Modulus-1.

Designing with the Core

This section includes guidelines and additional information to facilitate designing with the core.

General Design Guidelines

Use the Example Design

Each instance of the DDS Compiler core created by the Vivado design tool is delivered with an example design that can be implemented in a device and then simulated. This design can be used as a starting point for your own design or can be used to sanity-check your application in the event of difficulty. See the Example Design content for information about using and customizing the example designs for the core.

Registering Signals

To simplify timing and increase system performance in a programmable device design, keep all inputs and outputs registered between the user application and the core. This means that all inputs and outputs from the user application should come from, or connect to, a flip-flop. While registering signals might not be possible for all paths, it simplifies timing analysis and makes it easier for the AMD tools to place and route the design.

Recognize Timing Critical Signals

The constraints provided with the example design identify the critical signals and timing constraints that should be applied.

Make Only Allowed Modifications

You should not modify the core. Any modifications can have adverse effects on system timing and protocol compliance. Supported user configurations of the core can only be made by selecting the options in the customization IP dialog box when the core is generated.

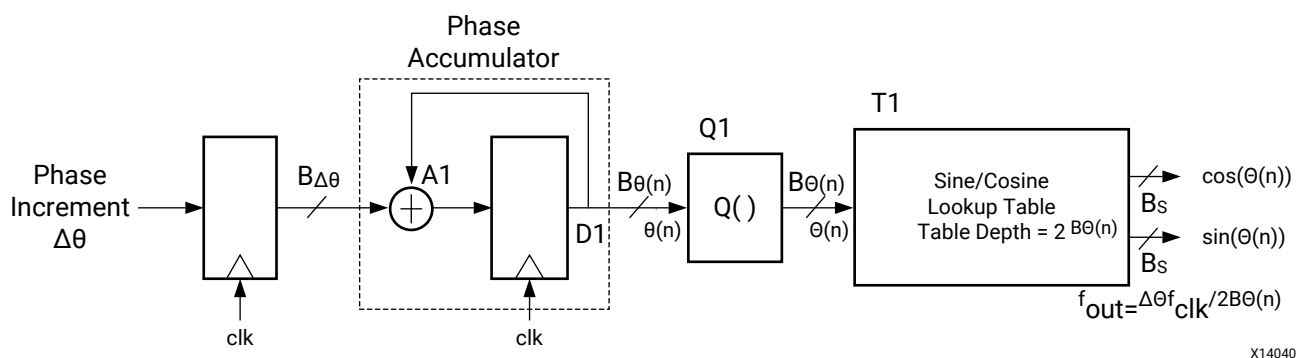
General Description

Direct digital synthesizers (DDS), or numerically controlled oscillators (NCO), are important components in many digital communication systems. Quadrature synthesizers are used for constructing digital down and up converters and demodulators. They are also used for implementing various types of modulation schemes, including PSK (phase shift keying), FSK (frequency shift keying), and MSK (minimum shift keying). A common method for digitally generating a complex or real valued sinusoid employs a lookup table scheme. The lookup table stores samples of a sinusoid. A digital integrator is used to generate a suitable phase argument that is mapped by the lookup table to the desired output waveform. A simple user interface accepts system-level parameters such as the desired output frequency and spur suppression of the generated waveforms.

Theory of Operation

The standard mode of the DDS Compiler core uses phase truncation, as shown in the following figure.

Figure 3: Phase Truncation DDS (Simplified View of the DDS Core)



The integrator (components D1 and A1) computes a phase slope that is mapped to a sinusoid (possibly complex) by the lookup table T1. The quantizer Q1, which simply truncates, accepts the high-precision phase angle $\theta(n)$ and generates a lower precision representation of the angle denoted as $\Theta(n)$ in the preceding figure. This value is presented to the address port of a lookup table that performs the mapping from phase-space to time.

The fidelity of a signal formed by recalling samples of a sinusoid from a lookup table is affected by both the phase and amplitude quantization of the process. The depth and width of the lookup table affect the phase angle resolution and the amplitude resolution of the signal, respectively. See [Spectral Purity Considerations](#) for more details.

Direct digital synthesizers use an addressing scheme with an appropriate lookup table to form samples of an arbitrary frequency sinusoid. If an analog output is required, the DDS presents these samples to a digital-to-analog converter (DAC) and a low-pass filter to obtain an analog waveform with the specific frequency structure. Of course, the samples are also commonly used directly in the digital domain. The lookup table traditionally stores uniformly spaced samples of a cosine and a sine wave. These samples represent a single cycle of a prototype complex sinusoid of length $N = 2^{B_{\Theta(n)}}$ and correspond to specific values of the sinusoid argument $\Theta(n)$ as follows, where n is the time series sample index:

$$\Theta(n) = n \frac{2\pi}{N}$$

Quarter-wave symmetry in the basis waveform can be exploited to construct a DDS that uses shortened tables. In this case, the two most significant bits of the quantized phase angle $\Theta(n)$ are used to perform quadrant mapping. This implementation results in a more resource efficient implementation because the memory requirements are minimized, offering either fewer FPGA block RAMs or reduced distributed memory. Based on the core customization parameters, the DDS core automatically employs quarter-wave or half-wave symmetry when appropriate.

For shallow tables, FPGA logic resources are actually minimized by storing a complete cycle. No design decisions are required in this context. The AMD adaptive computing tools always produce the smallest core possible.

The rasterized mode of operation of the DDS does not truncate the accumulated phase. Rasterized operation is intended for configurations where the desired frequency is a rational fraction of the system clock (output frequency = system frequency * N/M , where $0 < N < M$). Values of M from 9 to 16384 are supported. The SIN/COS LUT is configured accordingly for values from 0 to $M-1$, which describe a full circle. Because there is no phase truncation in the rasterized mode of operation, there is no need for dither or Taylor correction because these mitigate the effects of phase truncation. In rasterized operation, the phase noise is significantly reduced. Therefore, the output phase angle resolution and amplitude resolution are determined by the LUT table output width alone. In rasterized mode, quadrant symmetry is exploited where applicable to reduce memory use.

Output Frequency

This section covers output frequency when using the core in standard mode or rasterized mode.

Standard Mode of Operation

The output frequency, f_{out} , of the DDS waveform is a function of the system clock frequency, f_{clk} , the phase width (that is, the number of bits ($B_{\theta(n)}$) in the phase accumulator), and the phase increment value, $\Delta\theta$. The output frequency in Hertz is defined by:

$$f_{out} = \frac{f_{clk}\Delta\theta}{2^{B_{\theta(n)}}}$$

For example, if the DDS parameters are:

- $f_{clk} = 120 \text{ MHz}$
- $B_{\theta(n)} = 10$
- $\Delta\theta = 12$ (decimal)

Then the output frequency is calculated as follows:

$$\begin{aligned} f_{out} &= \frac{f_{clk}\Delta\theta}{2^{B_{\theta(n)}}} \text{ Hz} \\ &= \frac{120 \times 10^6 \times 12}{2^{10}} \\ &= 1.406250 \text{ MHz} \end{aligned}$$

The phase increment value $\Delta\theta$ required to generate an output frequency f_{out} Hz is:

$$\Delta\theta = \frac{f_{out} 2^{B_{\theta(n)}}}{f_{clk}}$$

If the DDS core is time-division multiplexed to do multiple channels, the effective clock frequency per channel is reduced. For C channels, the phase increment required is:

$$\Delta\theta = \frac{C f_{out} 2^{B_{\theta(n)}}}{f_{clk}}$$

Rasterized Mode of Operation

The output frequency f_{out} of the DDS waveform for a single channel configuration is a function of the system clock frequency f_{clk} , the modulus M , and the phase increment value $\Delta\theta$. The output frequency in Hertz is defined by:

$$f_{out} = \frac{f_{clk} \Delta\theta}{M}$$

For example, if the DDS parameters are:

- $f_{clk} = 120 \text{ MHz}$
- $M = 1000$
- $\Delta\theta = 12 \text{ (decimal)}$

Then the output frequency is calculated as follows:

- $f_{out} = 120 \times 106 \times 12 / 1000$
- $f_{out} = 1.44 \text{ MHz}$

The phase increment value $\Delta\theta$ required to generate an output frequency f_{out} Hz is:

$$\Delta\theta = \frac{f_{out} M}{f_{clk}}$$

If the DDS core is time-division multiplexed to do multiple channels, then the effective clock frequency per channel is reduced. For C channels, the phase increment required is:

$$\Delta\theta = \frac{C f_{out} M}{f_{clk}}$$

Frequency Resolution

This section covers frequency resolution when using the core in standard mode or rasterized mode.

Standard Mode of Operation

The frequency resolution Δf of the synthesizer is a function of the clock frequency and the number of bits $B_{\theta(n)}$ employed in the phase accumulator. The frequency resolution can be determined using:

$$\Delta f = \frac{f_{clk}}{2^{B_{\theta(n)}}}$$

For example, for the following DDS parameters:

- $f_{clk} = 120 \text{ MHz}$
- $B_{\Theta(n)} = 32$

the frequency resolution is:

$$\begin{aligned}\Delta f &= \frac{f_{clk}}{2^{B_{\Theta(n)}}} \\ &= \frac{120 \times 10^6}{2^{32}} \\ &= 0.0279396 \text{ Hz}\end{aligned}$$

In the time-division multichannel case, the frequency resolution is improved by the number of channels, as follows:

$$\Delta f = \frac{f_{clk}}{2^{B_{\Theta(n)}} C}$$

Rasterized Mode of Operation

The frequency resolution Δf of the synthesizer is a function of the clock frequency and the modulus. The frequency resolution can be determined using:

$$\Delta f = \frac{f_{clk}}{M}$$

For example, for the following DDS parameters:

- $f_{clk} = 120 \text{ MHz}$
- $M = 1000$

The frequency resolution is:

- $= 120 \times 10^6 / 1000$
- $= 120 \text{ kHz}$

In the time-division multichannel case, the frequency resolution is improved by the number of channels, as follows:

$$\Delta f = \frac{f_{clk}}{MC}$$

Phase Increment

For standard mode, phase increment values in the range 0 to 2^{N-1} describes the range $[0,360]^\circ$ (where N is the number of bits in the phase accumulator). For rasterized mode, phase increment values must be considered unsigned due to the internal implementation. The phase increment values $[0 \text{ to Modulus}-1]$ describe the range $[0,360]$.

The phase increment term $\Delta\theta$ defines the synthesizer output frequency. Consider a standard DDS with the following parameterization:

$$f_{clk} = 100 \text{ MHz}$$

$$B_{\theta(n)} = 18$$

To generate a sinusoid with frequency $f_{out} = 19 \text{ MHz}$, the required phase increment is:

$$\begin{aligned}\Delta\theta &= \frac{f_{out} 2^{B_{\theta(n)}}}{f_{clk}} \\ &= \frac{19 \times 10^6 \times 2^{18}}{100 \times 10^6} \\ &= 49807.36\end{aligned}$$

This value must be truncated to an integer giving the following actual frequency:

$$\begin{aligned}f_{out} &= \frac{\Delta\theta f_{clk}}{2^{B_{\theta(n)}}} \\ &= \frac{49807 \times 100 \times 10^6}{2^{18}} \\ &= 18.9998627 \text{ MHz}\end{aligned}$$

Consider a DDS in rasterized mode with the following parameterization:

- $F_{clk} = 100 \text{ MHz}$
- $M \text{ (Modulus)} = 1536$

To generate a sinusoid with frequency $f_{out} = 19 \text{ MHz}$, the required phase increment is:

$$\Delta\theta = \frac{f_{out}M}{f_{clk}}$$

$$\Delta\theta = \frac{19 \times 10^6 \times 1536}{100 \times 10^6}$$

$$\Delta\theta = 291.84$$

The closest integer value to this is 292, giving the following actual frequency:

$$f_{out} = \frac{f_{clk}\Delta\theta}{M}$$

$$f_{out} = \frac{292 \times 100 \times 10^6}{1536}$$

$$f_{out} = 19.0104167 \text{ MHz}$$

Spectral Purity Considerations

This section covers spectral purity considerations when using the core in standard mode or rasterized mode.

Standard Mode of Operation

The fidelity of a signal formed by recalling samples of a sinusoid from a lookup table is affected by both the phase and amplitude quantization of the process. The depth and width of the lookup table affect the phase angle resolution and the amplitude resolution of the signal, respectively. These resolution limits are equivalent to time base jitter and amplitude quantization of the signal and add spectral modulation lines and a white broad-band noise floor to the signal spectrum.

In conjunction with the system clock frequency, the phase width determines the frequency resolution of the DDS. The accumulator must have a sufficient field width to span the desired frequency resolution. For most practical applications, a large number of bits are allocated to the phase accumulator to satisfy the system frequency resolution requirements. By way of example, if the required resolution is 1 Hz and the clock frequency is 100 MHz, the required width of the accumulator is:

$$B_{\theta(n)} = \left\lceil \log_2 \left(\frac{f_{clk}}{\Delta f} \right) \right\rceil$$

$$? = \left\lceil \log_2 \left(\frac{100 \times 10^6}{1} \right) \right\rceil$$

$$\lceil \cdot \rceil = \lceil 26.5754 \rceil$$

$$\lceil \cdot \rceil = 27 \text{ bits}$$

where $\lceil \cdot \rceil$ denotes the ceiling operator. Due to excessive memory requirements, the full precision of the phase accumulator cannot be used to index the sine/cosine lookup table. A quantized (or truncated) version of the phase angle is used for this purpose. The block labeled Q1 in the phase truncation DDS, [Figure 3](#), performs the phase angle quantization. The lookup table can be located in block or distributed memory.

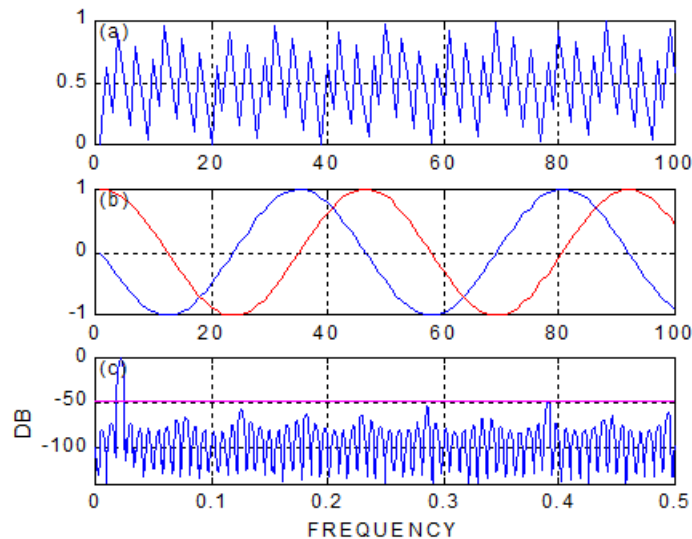
Quantizing the phase accumulator introduces time base jitter in the output waveform. This jitter results in undesired phase modulation that is proportional to the quantization error, as shown by the following:

$$\begin{aligned}\Theta(n) &= \theta(n) + \delta\theta \\ e^{j\Theta(n)} &= e^{j[\theta(n) + \delta\theta(n)]} = e^{j\theta(n)} e^{j\delta\theta(n)} \\ e^{j\Theta(n)} &\approx e^{j\theta(n)} [1 + j\delta\theta(n)] \\ &\approx e^{j\theta(n)} + j\delta\theta(n) e^{j\theta(n)}\end{aligned}$$

The following figure shows the lookup table addressing error, complex output time-series, and the spectral domain representation of the output waveform produced by the DDS structure shown in [Figure 3](#). The normalized frequency for this signal is 0.022 Hz, which corresponds to phase accumulation steps of 7.92° per output sample. The angular resolution of the 256-point lookup table is 360/256 or 1.40625° per address, which is equivalent to 7.92/1.40625 or 5.632 addresses per output sample. Because the address must be an integer, the fractional part is discarded and the resultant phase jitter is the cause of the spectral artifacts.

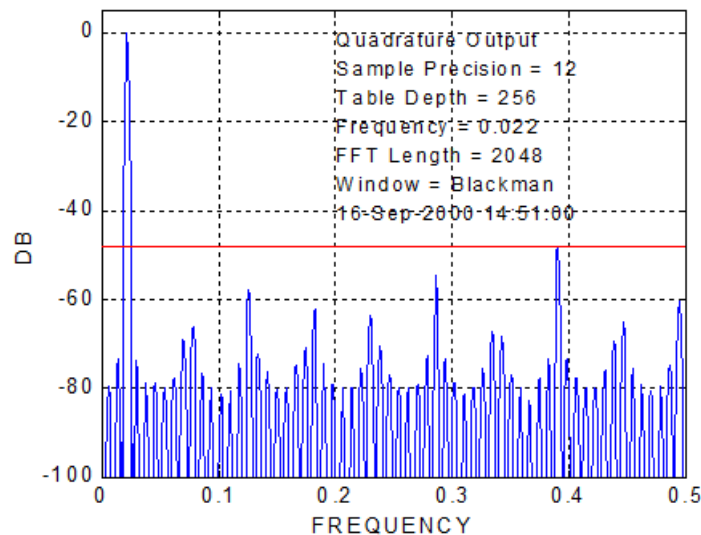
- **(a):** Phase angle addressing error
- **(b):** Complex output time series
- **(c):** Output spectrum

Figure 4: Phase Truncation DDS ($f_{out} = 0.022$ Hz, Table Depth = 256 12-Bit Precision Samples)



The following figure provides an exploded view of the spectral plot in the preceding figure.

Figure 5: Exploded View of Phase Truncation DDS ($f_{out} = 0.022$ Hz, Table Depth = 256 12-Bit Precision Samples)



Two observations related to the phase jitter structure level can be made. First, observe that the fractional part of the address count is a periodic (sawtooth) error sequence, which is responsible for the harmonic rich (and aliased) low-level phase modulation evident in the preceding figure. Also, the peak distortion level due to incidental phase modulation is approximately 48 dB below the desired signal level, which is consistent with 6 dB/bit of address space. Put another way, if S

dB of spur suppression is required in the output waveform, as referenced to the 0 dB primary tone, the DDS lookup table must support the dB of spur suppression divided by 6 (where 6 is the dB/bit off address space). For example, if $S = 70$ dB, which means that the highest spur is 70 dB below the main signal, the minimum number of address bits for the lookup table is the integer ceiling of 70 divided by 6, which is 12 bits; that is, a 4096-deep table.

The following figures demonstrate the performance of a similar DDS to the one presented in Figure 4, but in this example, 16-bit precision output samples have been used. Observe that the highest spur is still at the -48 dB level, and allocating four additional bits to the output samples has not contributed to any further spur reduction. For a phase truncation DDS, the only option to further reduce the spur levels is to increase the depth of the lookup table.

- (a): Phase angle addressing error
- (b): Complex output time series
- (c): Output spectrum

Figure 6: Phase Truncation DDS ($f_{\text{out}} = 0.022$ Hz, Table Depth = 256 16-Bit Precision Samples)

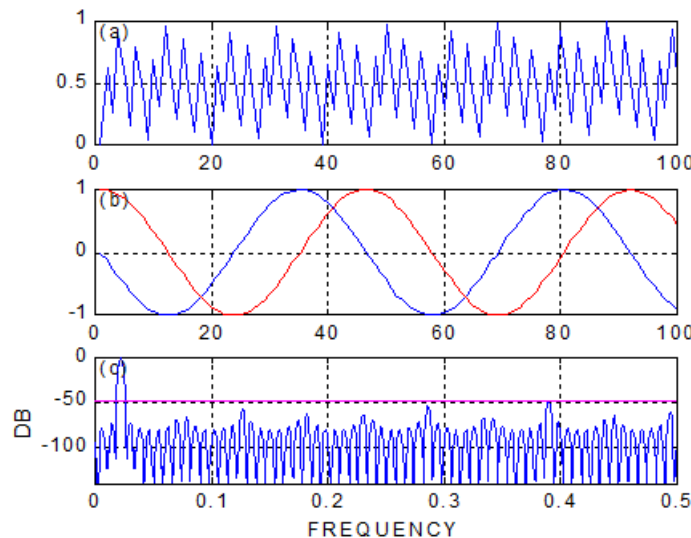
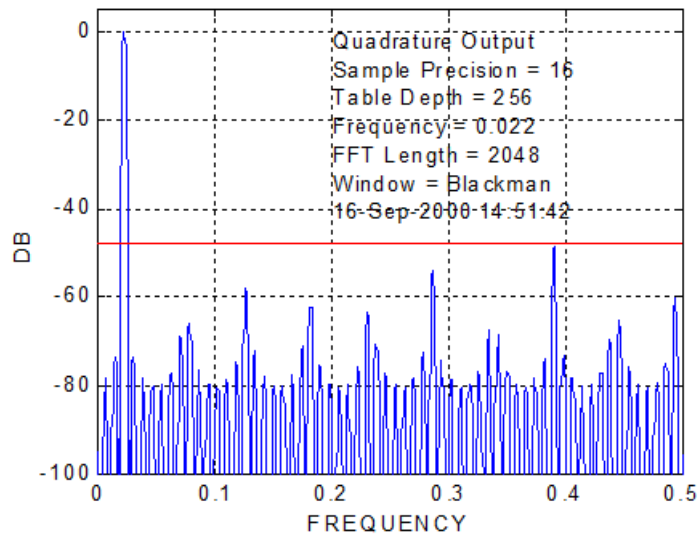


Figure 7: Exploded View of Phase Truncation DDS ($f_{\text{out}} = 0.022$ Hz, Table Depth = 256 16-Bit Precision Samples)



Rasterized Mode of Operation

In rasterized mode, the phase signal from the Phase Generator can be expressed exactly as an integer due to its rational fraction relationship to the system clock and the fact that no truncation of the phase vector occurs. The fidelity of the SINE/COSINE signals is therefore due only to the accuracy and precision of the SINE/COSINE table entries. Unlike the standard mode, there is no time base jitter. The effects of time base jitter described in [Standard Mode of Operation](#) do not affect a rasterized configuration.

Because there is no truncation of the phase vector from the phase accumulator and all possible values of phase have corresponding entries in the SINE/COSINE table, there is no need to compensate for or mitigate the phase error because there is no phase error. Dither and Taylor series correction are two techniques to reduce the effect of phase error. Because they are not required, they are not offered in rasterized mode.

The expected Spurious Free Dynamic Range of a rasterized design is 6 dB per bit of the output data field.

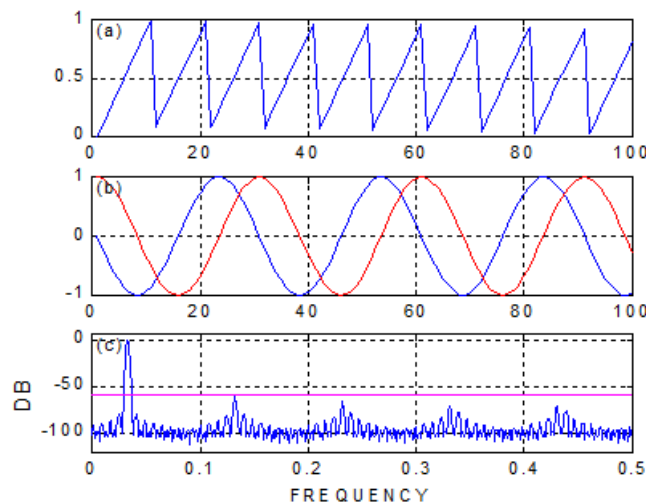
Phase Dithered DDS

Note: Phase dithering is only an option in standard mode of operation.

In the phase truncation DDS architecture shown in Figure 3, the quantizer Q1 introduces a phase error in the phase slope by discarding the least significant part of the high-precision phase accumulator. The phase error due to the discarded fractional part of the address count is a periodic series which results in an undesired spectral line structure. The following figure provides an example of this process for a DDS with a table depth $N = 1024$ and table sample precision of 16 bits.

- (a): Phase error generated by taking the difference between the quantizer input and output signals
- (b): Output time series
- (c): Signal output spectrum

Figure 8: DDS Plots (1024 Deep Lookup Table, 16-Bit Samples, Output Frequency 0.333 Hz)



Observe in (a) the periodic sawtooth structure of the phase error signal. The line spectrum associated with this correlated error sequence is impressed on the final output waveform and results in spectral lines in the synthesizer output spectrum. These spurious components can be clearly seen in (c).

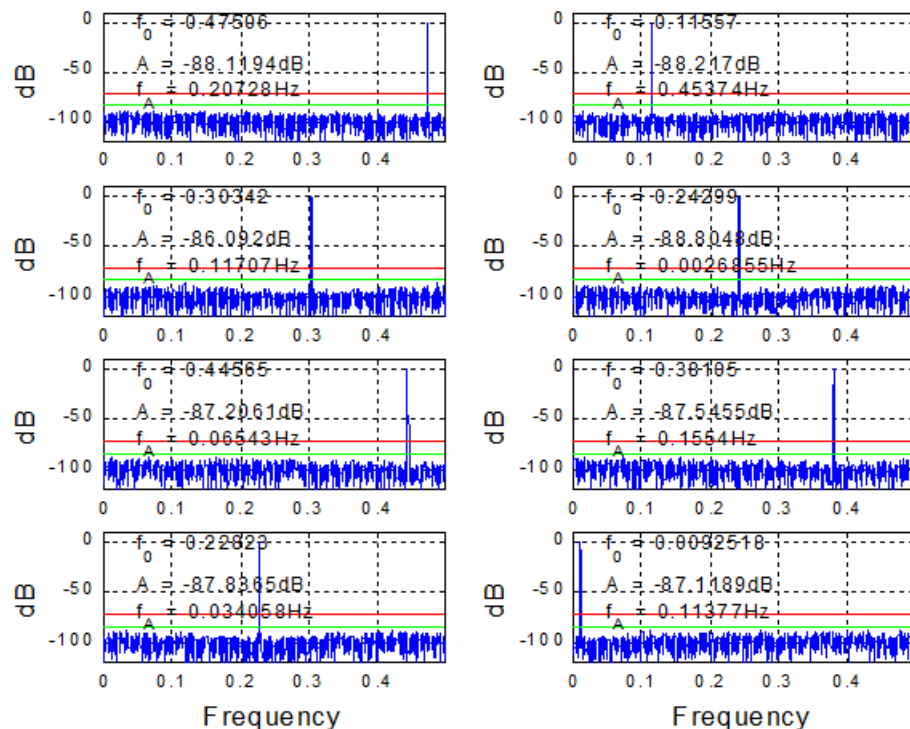
This structure can be suppressed by breaking up the regularity of the address error with an additive randomizing signal. This randomizing sequence, called *dither*, is a noise sequence, with variance approximately equal to the least significant integer bit of the phase accumulator. The dither sequence is added to the high-precision accumulator output prior to quantization by Q1.

The dithered DDS supplies, approximately, an additional 12 dB of spurious free dynamic range (SFDR) in comparison to a phase truncation design. This is achieved by spreading the spectral energy of the phase error signal. The additional logic resources required to implement the dither sequence generator are not significant.

To provide S dB of spur suppression using a phase truncation DDS, as referenced to the 0 dB primary tone, the internal lookup table must support at least $\lceil S/6 \rceil$ address bits. To achieve this same performance using the dithered architecture requires two fewer address bits, minimizing the number of block RAMs (or logic slices for a distributed memory implementation) used in the FPGA implementation. In summary, for a dithered DDS implementation, the number of address bits needed to support dB spur suppression is equal to $\lceil S/6 \rceil - 2$.

The following two figures provide the results for several dithered DDS simulations. The first figure shows eight simulations for a complex dithered DDS employing a table depth $N = 4096$ and 16-bit precision samples. For each plot the output frequency is different and is annotated on the plot. A phase truncation design would typically generate output spurs 72 dB below the output frequency, independent of the actual value of the output frequency. Indicated on each of the plots by the parameter A is the peak spur level achieved for the simulation. The eight spurs are -88.12, -88.22, -86.09, -88.80, -87.21, -87.55, -87.83, -87.12 dB below the output frequency. The worst case value of -86.09 is 14.09 dB better than a similarly configured phase truncation DDS.

Figure 9: Dithered DDS Simulations (DDS Configuration $N = 4096$, $B_s = 16$)

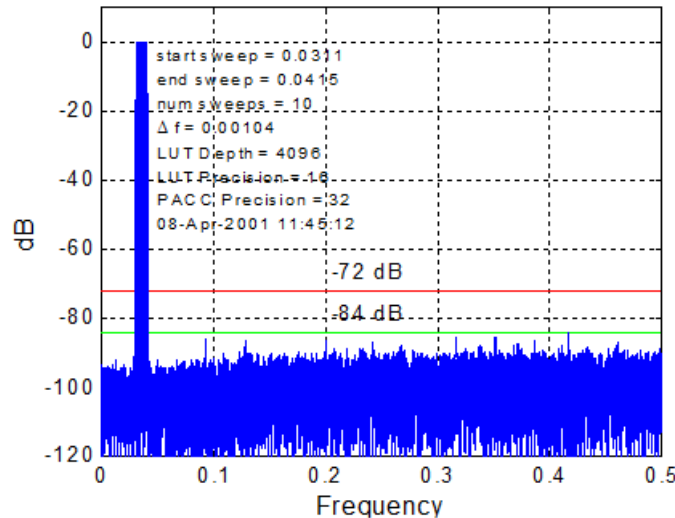


The eight plots are spectral domain representations for eight different output frequencies. Each plot is annotated with the peak spur.

To achieve this same SFDR by extending the table length of a phase truncation design would require increasing the table depth by more than a factor of four.

The following figure provides one more dithered DDS simulation where the output frequency is swept over a band of frequencies. The spectrum for each discrete tone in the sweep band is overlaid to construct the final plot. The sweep start frequency, end frequency, number of tones in the sweep, and DDS configuration are annotated on the plot.

Figure 10: Example Plot for Dithered DDS Simulation with Frequency Sweep



In the preceding figure, the synthesized signal is swept over a range of frequencies starting from 0.0311 to 0.0415 Hz. There are 10 tones in the sweep separated in frequency by 0.00104 Hz. In this example, the phase truncation DDS would produce peak spurs at -72 dB with respect to the 0 dB primary signal. The dithered DDS provides approximately 12 dB better performance with the peak spur -84 dB below the output signal.

A further advantage of the dithered DDS is that the spectral line structure present in a phase truncation design is removed and the out-of-band signal is significantly whitened. This white broadband noise floor is more desirable than the line structured spectrum. In digital communication receivers that use a DDS for generating mixing signals for performing channelization functions, the spurs in a phase truncation DDS can act as low-level mixing tones and cause undesirable spectral contamination of the desired channel. For virtually all applications, the preferred implementation is the dithered DDS.

Taylor Series Corrected DDS

Note: Taylor series correction is only an option in standard mode of operation.

The phase dithered and phase truncation DDS have a quantizer Q1 that produces a lower precision $\Theta_{(n)}$ by discarding the fractional component of the high precision $\theta_{(n)}$. The reason for this quantization step is to keep the size of the lookup memory to a reasonable size. The trade-off is spectral purity. With the availability of DSP slices in FPGAs, it is now practical to use the previously discarded fractional bits to calculate corrections that can be added to the lookup table values to produce outputs with very high SFDR.

Figure 11 through Figure 14 show the simulation results of four different Taylor series corrected DDS simulations. The Taylor series corrected architecture in this example uses a table depth $N = 4096$ and 18-bit precision samples. However, the precision at the output of the feed-forward error processor is 20 bits. For each plot, the output frequency is different and annotated directly on the plot. A similarly configured phase truncation DDS would produce spurs at -72 dB and a phase dithered DDS at -84 dB. The peak spurs for the four plots are -118.25 , -118.13 , -118.10 , and -118.17 dB below the output frequency.

Figure 15 shows a swept frequency Taylor series corrected DDS. The starting frequency for this example is 0.0313 Hz, the final frequency is 0.0813, and there are 100 tones in the sweep. Using this configuration, a phase truncation DDS would produce peak spurs at approximately 72 dB below the output signal and a phase dithered DDS would produce peak spurs at approximately 84 dB below the output signal. As shown in the plot, the Taylor series corrected DDS produced spurs that are all the way down to 118 dB below the output signal. This result is 34 dB better than the phase dithering DDS, 46 dB better than the phase truncation DDS, and still only consumes a single 18 Kb block RAM for the lookup storage. Figure 16 shows another frequency sweep simulation with 35 tones over a broader frequency range.

As shown in the plots, linear correction of the RAM values can extend the SFDR to 118 dB using only a single block RAM and three multipliers. To achieve SFDR beyond 118 dB, it is necessary to deepen the RAM or to use quadratic correction (an extra term of the Taylor series). Because the RAM size would double for each additional 6 dB, the DDS Compiler uses quadratic correction to achieve SFDR values of up to 150 dB. Introducing the extra term of the Taylor series expansion of Sine or Cosine requires an additional multiplier per sine and cosine output and an additional block RAM to both scale and square the phase error.

Optimization of Memory Usage

The Taylor Series correction implementation in the DDS Compiler core typically results in an SFDR higher than that requested in order to guarantee SFDR. This results in extra block RAMs for values of SFDR above 102 dB. However, in many cases, depending on the phase increment values used, a specified SFDR target value of 102 dB provides higher SFDR, but with one 18 K block RAM.

Figure 11: Taylor Series Corrected DDS – Single-Tone Test, $f_0 = 0.0092518$

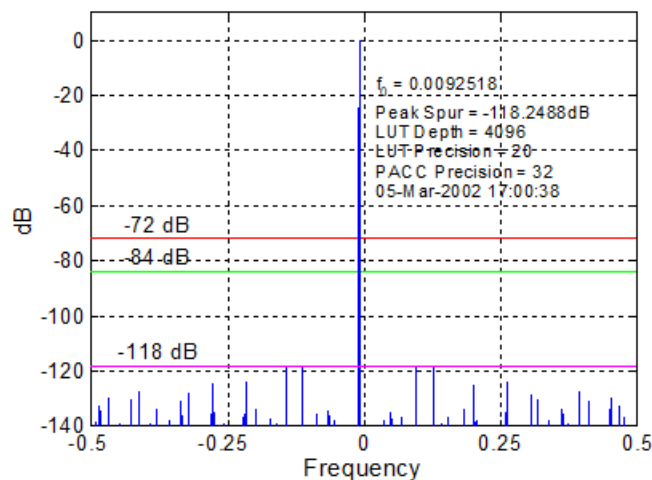


Figure 12: Taylor Series Corrected DDS – Single-Tone Test, $f_0 = 0.22823$

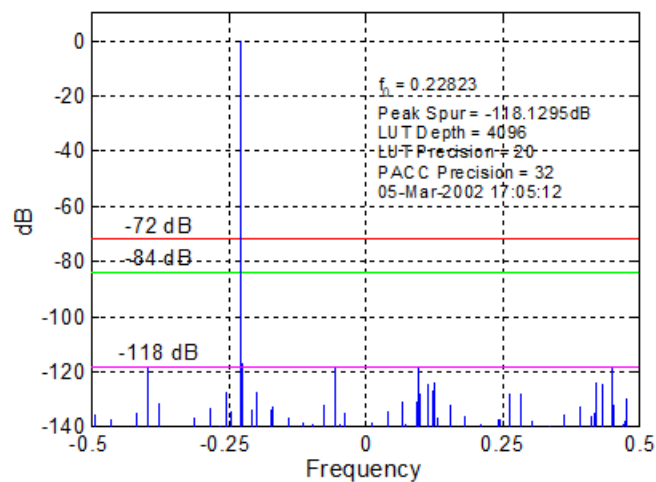


Figure 13: Taylor Series Corrected DDS – Single-Tone Test, $f_0 = 0.30342$

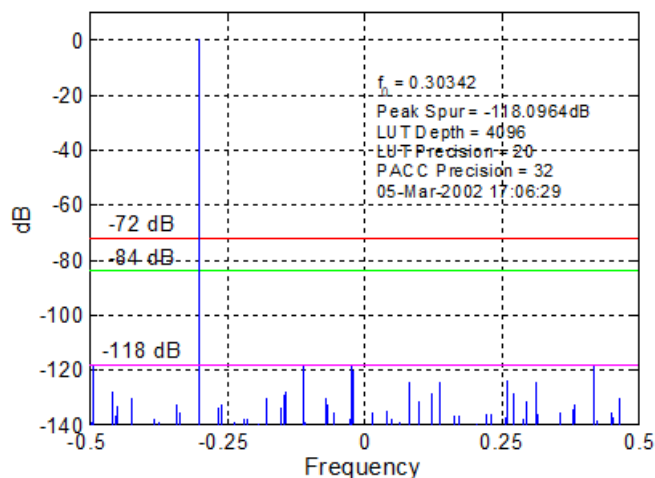


Figure 14: Taylor Series Corrected DDS – Single-Tone Test, $f_0 = 0.47506$

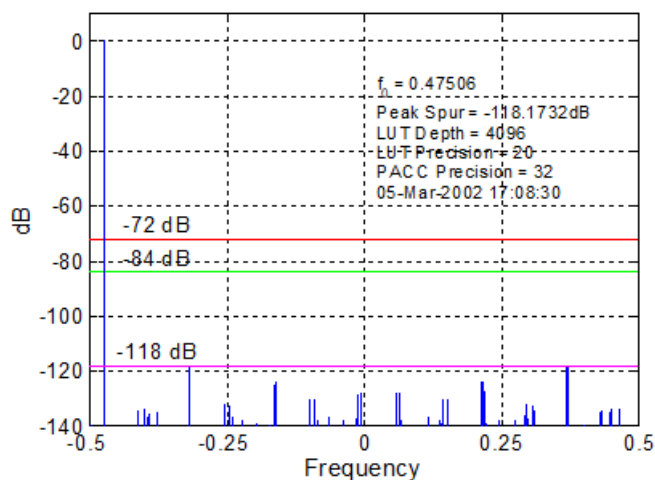


Figure 15: Taylor Series Corrected DDS – Frequency Sweep Simulation, 100 Tones

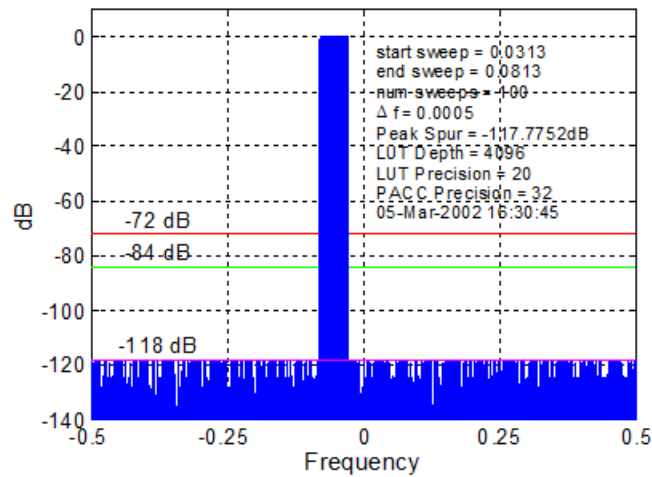
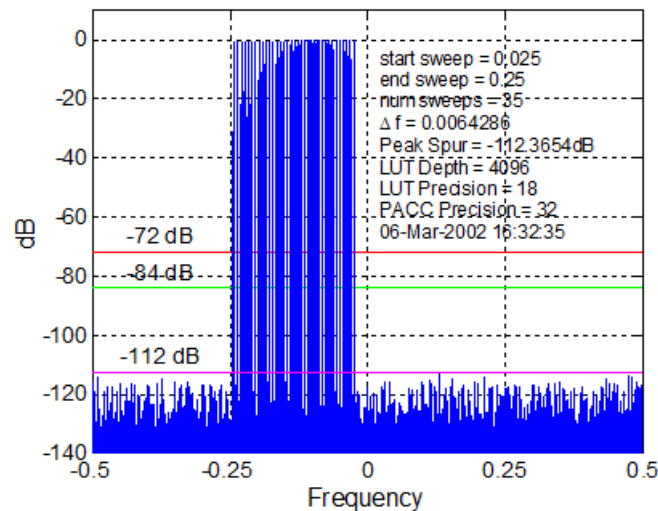


Figure 16: Taylor Series Corrected DDS – Frequency Sweep Simulation, 35 Tones



Multichannel

When configured for more than one channel, the DDS, or Phase Generator part, generates outputs for each channel in a time-multiplexed fashion. As such, the output for a particular channel occurs every N-cycles, where N is the number of channels selected when the core was customized. The outputs for channel 0 are given first.

Design Examples

The DDS Compiler user interface accepts system-level parameters instead of low-level parameters such as the width of the phase accumulator or width of the phase angle. Because of this, all preceding requirements can be entered into the Vivado IDE directly without having to calculate low-level core details. The user interface in the Vivado IDE also provides feedback on the hardware parameters so the translation of system-level parameters to low-level parameters can be seen. Alternatively, hardware parameters can be entered directly.

Example 1

In standard mode, with a single-channel DDS, a 1 MHz system clock, and a frequency resolution of 1 Hz, phase width is 20 bits. To synthesize an output of 23.4 kHz, an output frequency value of 0.0234 MHz must be entered into the user interface, which then returns a value of 5FD8 in hexadecimal, which is 24536 in decimal.

This gives a synthesized frequency of $24536/2^{20} * 1 \text{ MHz} = 23399.35 \text{ Hz}$.

If the application requires this frequency to be modulated by one of eight phase offsets, the phase offset bus need only have 3-bit precision, but these must be the top 3 bits of the phase offset input. Hence, the phase offset of one eighth of a cycle would be entered as 0.125 in the user interface. This returns a value of 20000 (hex). This could be entered on the 3-bit bus as 001 (binary).

Example 2 (DDS Requiring Negative Frequencies)

In standard mode, with a single-channel DDS, a 100 MHz system clock, and a frequency resolution of 1 Hz, the phase width is 25 bits. Frequencies of -3 MHz, -1 MHz, 1 MHz, and 3 MHz are required. F_s is the frequency per channel, which is system clock/number of channels: that is, 25 MHz. The negative frequencies alias to every F_s Hz. The legal range to enter in the user interface is 0 to F_s , so the entered frequencies for this example must be 22 MHz ($F_s - 3 \text{ MHz}$), 24 MHz ($F_s - 1 \text{ MHz}$), 1 MHz, and 3 MHz respectively.

Clocking

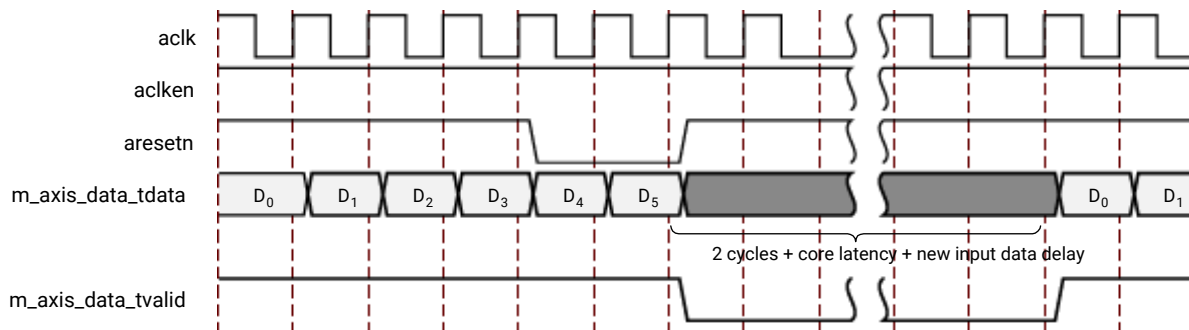
The DDS Compiler has only one clock signal input, `ac1k`. This is the system clock referred to in equations. There are no special timing or placement constraints. Because the DDS Compiler has only one clock, there are no clock domain crossing considerations.

Resets

The active-Low reset signal, `aresetn`, is registered within the core to ensure high performance. This signal has the requirement that it must be driven Low for a minimum of two cycles to guarantee that there is no violation of the AXI4-Stream protocol on the core outputs. The registering of reset appears as a delay in entering the reset state, and the signal states after reset differ depending on whether TREADY signals are present.

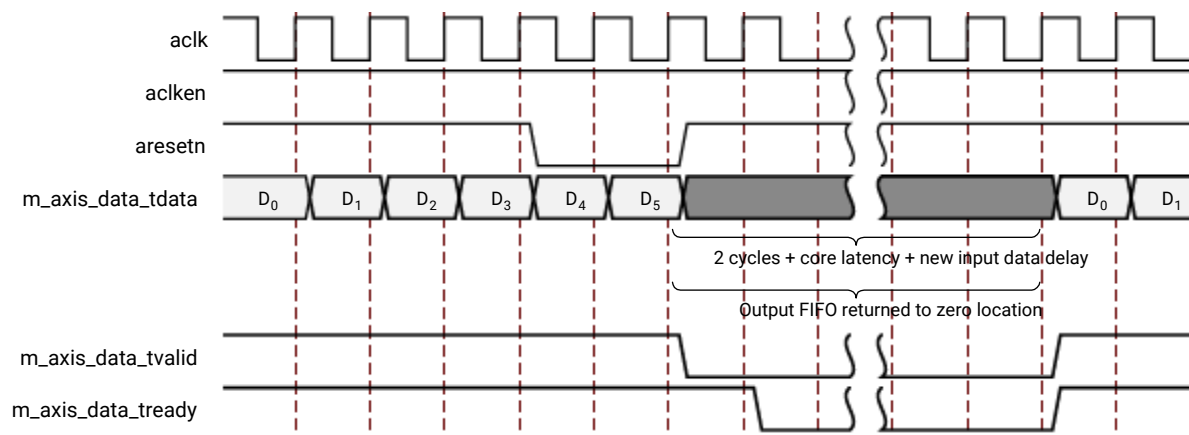
The following diagram shows the core entering reset when there are no TREADY signals, using a multichannel DDS as an example. The subscript on the valid data outputs indicates a channel number. The true behavioral model outputs X values for TDATA fields during reset. All channel payloads are qualified with their respective TVALID outputs.

Figure 17: Reset Behavior (no TREADY)



The following figure shows the core entering reset when TREADY is present, using a multichannel DDS as an example. The subscript on the valid data outputs indicates a channel number. The reset event forces the FIFOs within the core back into their initial state; no X value is produced by the VHDL RTL-based behavioral model in this case. Again, all channel payloads are qualified with their respective TVALID outputs.

Figure 18: Reset Behavior (with TREADY)



Protocol Description

Other than the event signals, all interfaces to and from the DDS Compiler are AXI4-Stream interfaces. The event signals are registered and asserted on detection of the event they describe.

AXI4-Stream Considerations

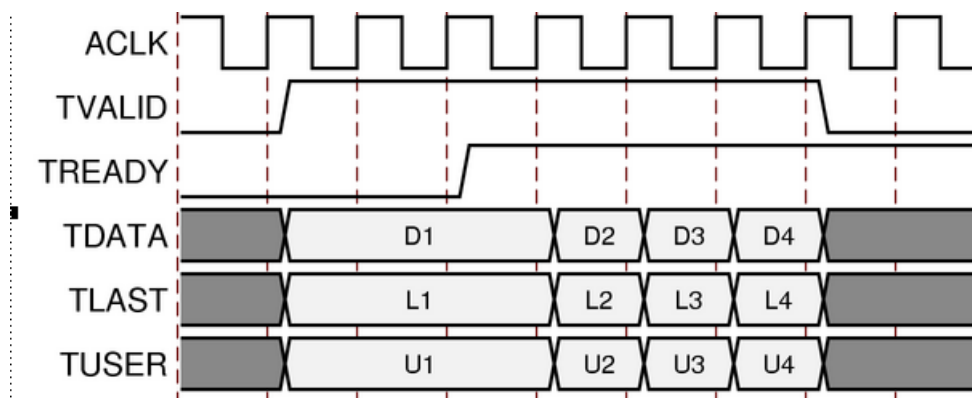
The conversion to AXI4-Stream interfaces brings standardization and enhances the interoperability of AMD LogiCORE™ IP solutions. Other than general control signals such as `aclk`, `aclken`, `aresetn`, and event indication outputs, all inputs and outputs of the DDS Compiler are conveyed on AXI4-Stream channels. A channel always consists of TVALID and TDATA, plus several optional ports and fields. In the DDS Compiler, the optional ports supported are TREADY, TLAST and TUSER. Together, TVALID and TREADY perform a handshake to transfer a message, where the payload is TDATA, TUSER and TLAST. The DDS Compiler operates on the operands contained in the TDATA field of the input channels and outputs the result in the TDATA field of the output channels. The DDS Compiler provides configuration options for the use of TUSER and TLAST.

Basic Handshake

The following figure shows the transfer of data in an AXI4-Stream channel. TVALID is driven by the source (master) side of the channel and TREADY is driven by the receiver (slave). TVALID indicates that the value in the payload fields (TDATA, TUSER, and TLAST) is valid. TREADY indicates that the slave is ready to receive data. When both TVALID and TREADY are asserted in a cycle, a transfer occurs. The master and slave set TVALID and TREADY respectively for the next transfer appropriately.

The DDS Compiler datapath channels (all except the CONFIG channel) can be configured to have no TREADY.

Figure 19: Data Transfer in an AXI4-Stream Channel



This is equivalent to setting TREADY for each of these channels permanently asserted. This inability to indicate backpressure simplifies the interface behavior and allows resource savings to be made. This mode is recommended when the system can be designed to ensure that full throughput (one sample per cycle) can be assured.

The AXI4 handshake shown in the preceding figure forbids the output of data until the downstream circuitry is ready. This allows for easy synchronization of circuits following power-up or reset without complicated latency calculations.

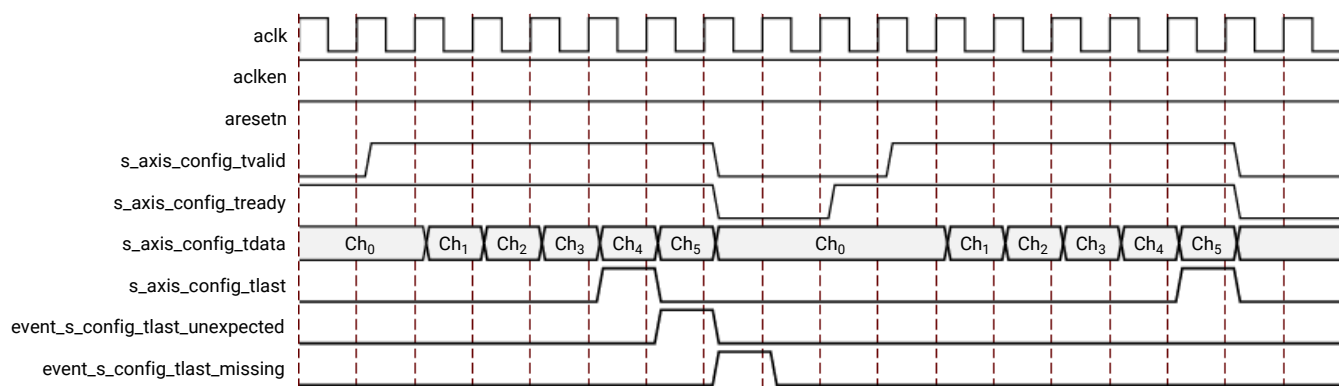
For more information on AXI4-Stream interfaces, see the *AMBA AXI4-Stream Protocol Specification* ([ARM IHI 0051A](#)).

CONFIG Channel

The CONFIG channel (`s_axis_config_t*`) replaces the programmatic interface of DDS Compiler v4.0. For the CONFIG channel, there is the concept of a vector. The vector in question is a complete set of values (PINC and/or POFF) for all channels. The CONFIG channel is non-blocking, which means that the other channels of the DDS Compiler do not wait upon data from the CONFIG channel. To program the CONFIG channel, N transfers must occur, where N is the number of channels. Each transfer contains the PINC and/or POFF values for each channel in sequence starting with channel 0. Only the last transfer, for channel (index N-1), must have TLAST asserted. Failure to do so causes either `event_s_config_tlast_missing` or `event_s_config_tlast_unexpected` outputs to be asserted for a cycle. The packet is only deemed to be received when complete. Only when it is completely received is it eligible to be used pending a synchronization event. Synchronization events are either when the TDM channel counter rolls over (vector framing) or when the input PHASE channel is configured to receive packet TLASTs and one such TLAST is received (packet framing).

The following figure illustrates programming CONFIG data for a six-channel DDS. In the first programming cycle, TLAST is incorrectly applied, and the event outputs trigger accordingly. The second programming cycle shows correct application of TLAST.

Figure 20: CONFIG Channel Programming



When the core is configured for single-channel operation, TLAST is not required and the pin is not present on the CONFIG channel.

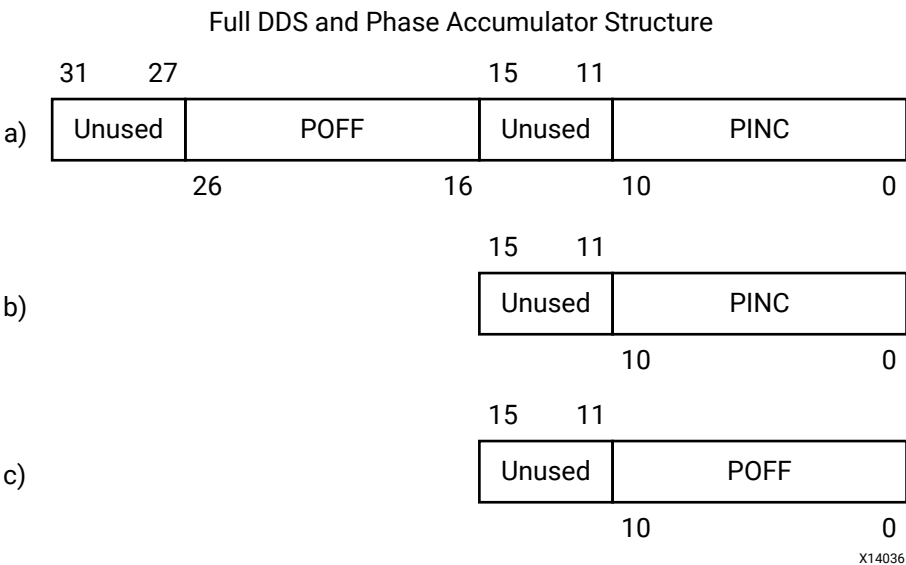
CONFIG Channel TDATA Structure

When the CONFIG channel is configured to supply both PINC and POFF values for each TDM channel, each field is sign-extended to fit a byte boundary, and then these byte-oriented fields are concatenated with PINC in the least significant positions. For example, for a phase width of 11 bits, PINC would occupy bits 10:0 and POFF would occupy 26:16. Thus `s_axis_config_tdata` would be 31:0 overall.

The following figure shows the structure for the widths in the preceding example for the following configurations:

- Both PINC and POFF are set to Programmable.
- PINC only is set to Programmable.
- POFF only is set to Programmable.

Figure 21: CONFIG Channel TDATA Structure



Input PHASE Channel

The input PHASE channel (`s_axis_phase_t*`) replaces the streaming interface (PINC_IN and POFF_IN) or PHASE_IN ports of the DDS Compiler v4.0. The input PHASE channel is intended for applications where the DDS Compiler is to perform a dynamic function such as phase or frequency modulation, where there is an output sample for each input sample. The fact that there is a one-to-one relationship between input and output means that back pressure applied to the output (TREADY deasserted) results in the deassertion of TREADY on the input PHASE channel (delayed according to internal buffer capacity). Likewise, a starvation of input data on the PHASE channel (deassertion of TVALID) propagates to become a deassertion of TVALID on the output channels.

When in rasterized mode, values of PINC, POFF and PHASE_IN must be constrained between 0 and Modulus-1 (inclusive). This corresponds to a full circle. Therefore, if negative values are required, add the Modulus to map these into the required range. For instance, with Modulus 360, if -90 is required, add 360 to get 270, which is in the required range. Event signals exist to detect invalid values of PINC, POFF and PHASE_IN input at runtime.

When the DDS Compiler is configured to have a Phase Accumulator, and either Phase Increment or Phase Offset is selected to be “Streaming”, the input PHASE channel interface exists. When the DDS Compiler is configured to be a SIN/COS LUT only, the PHASE_IN field is input on the TDATA bus of the input PHASE channel. These two configurations are mutually exclusive.

Input PHASE Channel TDATA Structure

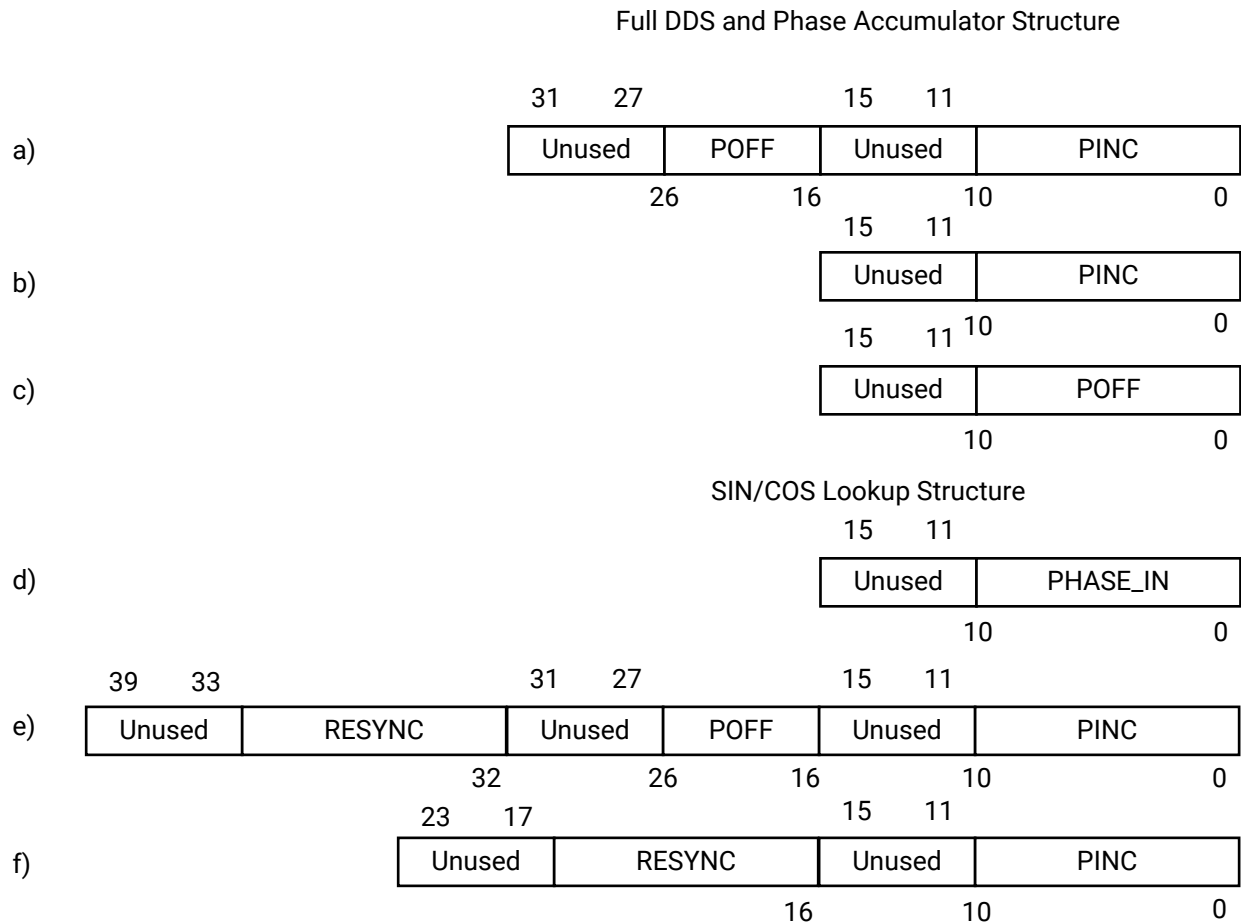
The two configurations in which the DDS Compiler can have an input PHASE channel are mutually exclusive. although there are four fields which can occur in the TDATA, all four cannot occur together.

When the DDS Compiler is configured to be a SIN/COS LUT only, the PHASE_IN field is mapped to `s_axis_phase_tdata`. The PHASE_IN field occupies a byte-oriented field in the least significant portion of the bus. The width of `s_axis_phase_tdata` is therefore the minimum multiple of eight bits required to accommodate the PHASE_IN width. Because this is an input, any additional bits required to achieve this byte orientation are ignored by the core and are optimized away during synthesis or mapping.

The following figure shows the structure of `s_axis_phase_tdata` where Phase_Width = 11 for the following configurations:

- Both PINC and POFF are set to Streaming.
- PINC only is set to Streaming.
- POFF only is set to Streaming.
- The DDS is configured to be SIN/COS LUT only.

Figure 22: Input PHASE Channel TDATA Structure



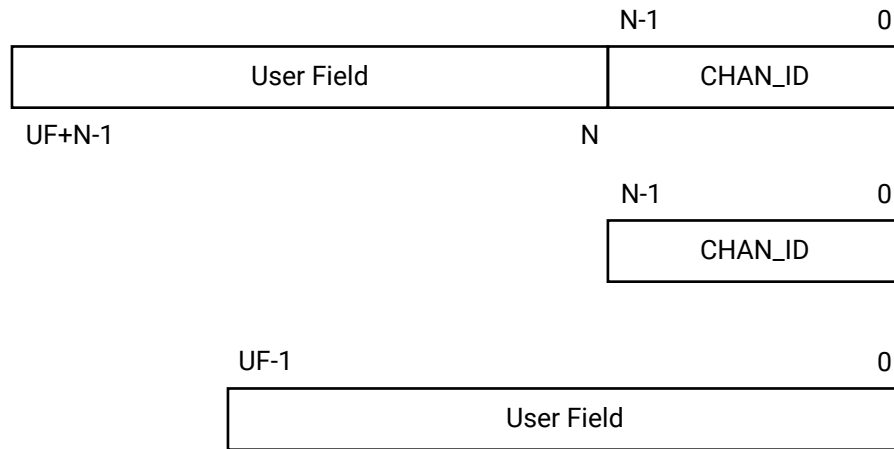
X13214

Input PHASE Channel TUSER Structure

The input PHASE channel can be configured to have no TUSER port, a user field or to carry the TDM channel index, or both a user field and TDM channel index. There is no byte orientation to these fields. The TDM channel index, if configured, has the minimum width required to describe the number of TDM channels. The width of the user field is determined by user selection from 1 to 256 bits. The two fields are concatenated with the TDM channel ID field in the least significant place. If only one field exists, it occupies the least significant bits of `s_axis_phase_tuser`.

The following figure shows the three possible combinations; both user field and `chan_id` field, `chan_id` field only, and user field only.

Figure 23: Input PHASE Channel TUSER Structure



$N = \text{ceil}(\log_2(\text{number of TDM channels}))$
 $UF = \text{User Field Width}$

X14038

Input PHASE Channel TLAST Options

The input PHASE channel can be configured to have no TLAST, to have a vector framing TLAST, or to have a packet framing TLAST.

When Vector Framing is selected, TLAST is expected to indicate the last channel in a TDM cycle of channels. If TLAST does not match the internal expectation of when TLAST should arrive, one of two event signals is asserted for a clock cycle.

When Packet Framing is selected, the core does not have any expectation of the timing of TLAST so the event signals are not present, but TLAST is conveyed to the output channels with the same latency as the TDATA input.

Output DATA Channel

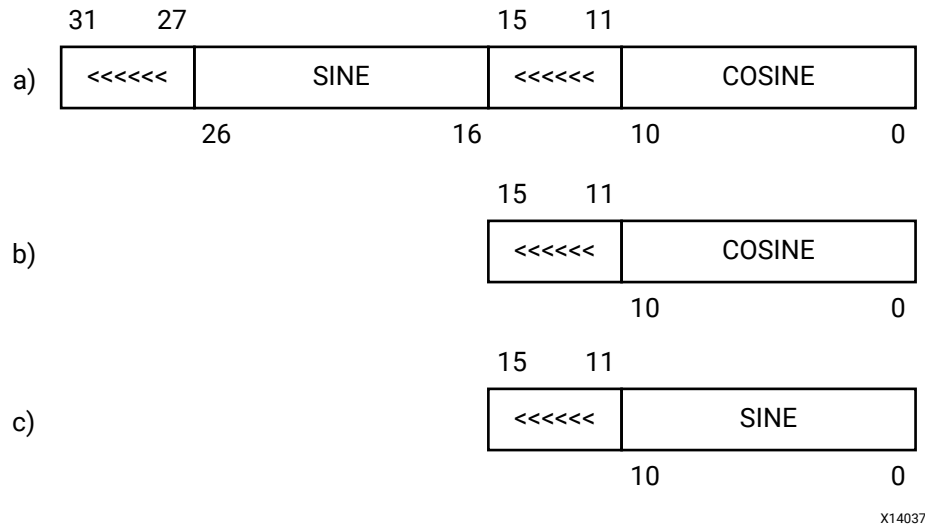
The output DATA channel exists whenever the DDS Compiler is configured to have a SIN/COS LUT. This channel replaces the SINE and COSINE outputs of DDS Compiler v4.0. These former outputs now exist as fields of `m_axis_data_tdata`.

Output DATA Channel TDATA Structure

The sine and cosine output fields are sign extended to the next byte boundary then concatenated, with cosine in the least significant portion, to create `m_axis_data_tdata`. If only one of sine or cosine is selected, it is sign extended and put into the least significant portion of `m_axis_data_tdata`.

The following figure shows the internal structure of TDATA for the three configurations: quadrature outputs, cosine only, and sine only. An 11-bit output has been shown in the diagram for example, sign extended to 16 bits. The <<<<< denotes sign extension.

Figure 24: Output DATA Channel TDATA Structure



Output DATA Channel TUSER Structure

The output DATA channel can be configured to have no TUSER field, or for TUSER to hold a user field, or a TDM channel index or both a user field and a TDM channel ID. When both user field and TDM channel ID are selected the fields are concatenated with the TDM channel ID in the least significant position. The TDM channel ID qualifies the fields in the TDATA bus for that transfer as belonging to the TDM channel described. See Figure 23 for the structure, because it is identical to the structure for the output DATA channel TUSER port.

The user field is not used nor interpreted by the core. It is provided as a service to allow the system designer to pass information through the core with latency identical to the main datapath (input PHASE channel to output channels). For instance, the user field could contain flags and other ancillary information irrelevant to the DDS, but relevant to some core downstream from the DDS.

Output DATA Channel TLAST Options

The output DATA channel can be configured to have no TLAST, to have vector framing, to have packet framing, or to have a TLAST that is configuration triggered.

When set to Vector Framing, TLAST is asserted for the transfer that contains the TDATA of the last TDM channel of a cycle of TDM channels (for example, channel 12 of 12).

When set to Packet Framing, the TLAST of the input PHASE channel is passed unaltered with latency equal to the latency of the main datapath. This is intended to be a service to the system designer, where TLAST might have a meaning which is irrelevant to the DDS, but relevant to some core downstream.

When set to Configuration Triggered, the TLAST is generated internally by the DDS rather than conveyed from the input PHASE channel. It is asserted on the last channel of a TDM cycle immediately before a configuration change is effected. In other words, if a configuration change is provoked using the CONFIG channel, TLAST is asserted on the last sample of the old configuration.

Output PHASE Channel

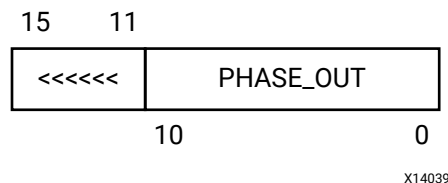
The output PHASE channel replaces the PHASE_OUT port of DDS Compiler v4.0. The PHASE_OUT port now exists as a field of `m_axis_phase_tdata`.

Output PHASE Channel TDATA Structure – Conventional DDS

The PHASE_OUT field is sign extended to the next multiple of eight bits and becomes `m_axis_phase_tdata`. For example, if PHASE_OUT is 20 bits, `m_axis_phase_tdata` is 24 bits wide [23:0], occupied by a sign extended PHASE_OUT.

The following figure shows this for an example width of 11 bits sign extended to 16 bits.

Figure 25: Output PHASE Channel TDATA Structure



Output PHASE Channel TDATA Structure – Rasterized DDS

The phase values in rasterized mode are positive integers in the range 0 to Modulus-1. The PHASE_OUT field is therefore padded with zeros up to the next multiple of eight bits, and becomes `m_axis_phase_tdata`.

Output PHASE Channel TUSER Structure

The TUSER field has the same configuration options as the output DATA channel, but the options are independent for the two output channels. For instance, one might be configured to have a user field while the other has a TDM channel ID field. Figure 23 is identical to the structure options for the output PHASE channel TUSER port.

Output PHASE Channel TLAST Options

The output PHASE channel uses the same TLAST setting as the output DATA channel.

Event Interface

To allow users to synchronize correctly to the internal channel counter, and flag errors when writing multichannel data to the CONFIG and/or PHASE channels, the DDS Compiler core provides several event outputs to indicate when unexpected conditions have occurred. Event outputs obey `aclken` and `aresetn` conditions, occur immediately, and are asserted for each cycle that a discrepancy is present.

If the event outputs are not required, they can be left unconnected and the associated logic is optimized away by the AMD adaptive computing tools.

CONFIG Channel Event Outputs

For multichannel configuration, the CONFIG channel expects a single pulse on `s_axis_config_tlast` to indicate the last sample in a sequence of channels (a “vector”). If `s_axis_config_tlast` is asserted when the core does not expect it to be, `event_s_config_tlast_unexpected` is asserted. If the `s_axis_config_tlast` pulse is not asserted with the configuration data of the last channel, `event_s_config_tlast_missing` is asserted.

PHASE Channel Event Outputs

When the PHASE channel is present with multiple channels, the PHASE channel expects a single pulse on `s_axis_phase_tlast` to indicate the last sample in a sequence of channels (a “vector”). If `s_axis_phase_tlast` is asserted when the core does not expect it to be, `event_s_phase_tlast_unexpected` is asserted. If the `s_axis_phase_tlast` pulse is not asserted with the data of the final channel, `event_s_phase_tlast_missing` is asserted.

When the DDS is configured for rasterized mode, both PINC and POFF values (fixed, programmable or streaming) are expected to be in the range 0 to Modulus-1. Values outside this range are not supported and can lead to erroneous output. The signals `event_pinc_invalid` and `event_poff_invalid` are asserted when a value outside the supported range is detected.

The `s_axis_phase_tuser` configuration options allow you to input a Channel ID value to facilitate synchronization with the internal channel counter. If this option is selected, the `event_s_phase_chanid_incorrect` output is present on the core and is asserted for every cycle where the input Channel ID on `s_axis_phase_tuser` does not match the current channel of the core.

Design Flow Steps

This section describes customizing and generating the core, constraining the core, and the simulation, synthesis, and implementation steps that are specific to this IP core. More detailed information about the standard AMD Vivado™ design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
- *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
- *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
- *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))

Customizing and Generating the Core

This section includes information about using AMD tools to customize and generate the core in the AMD Vivado™ Design Suite.

If you are customizing and generating the core in the Vivado IP integrator, see the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#)) for detailed information. IP integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, run the `validate_bd_design` command in the Tcl console.

You can customize the IP for use in your design by specifying values for the various parameters associated with the IP core using the following steps:

1. Select the IP from the IP catalog.
2. Double-click the selected IP or select the Customize IP command from the toolbar or right-click menu.

For details, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)) and the *Vivado Design Suite User Guide: Getting Started* ([UG910](#)).

Figures in this chapter are illustrations of the Vivado IDE. The layout depicted here might vary from the current version.

Configuration Tab

- **Configuration Options:** The full DDS or optionally the Phase Generator part or SIN/COS LUT part can be generated.
 - **Phase Generator and SIN/COS LUT:** DDS is provided by combining Phase Generator and SIN/COS LUT with an optional dither circuit.
 - **Phase Generator only:** Only the phase generator is provided.
 - **SIN/COS LUT only:** Only the SIN/COS LUT with optional Taylor series correction circuit is provided.
- **System Requirements:**
 - **System Clock:** The frequency at which the DDS core is clocked. The value provided influences architectural choices, and is used to calculate the value of phase increment from output frequency (it is the relative value of output frequency to system clock that specifies phase increment, and so doubling system clock while maintaining output frequency results in a doubling of phase increment).



IMPORTANT! *The specified clock rate might not be achievable by the final implementation, because this depends on the FPGA family and how much is being packed into the device.*

- **Number of Channels:** The DDS and phase generator can support up to 16 channels. The channels are time-multiplexed, which reduces the effective clock frequency per channel.
- **Mode of Operation:** The DDS supports standard mode, where the accumulated phase can be truncated before being used to access the SIN/COS LUT, or rasterized mode, which can be used when the desired frequencies and system clock are related by a rational fraction. See [Theory of Operation](#) for more details.
- **Modulus (rasterized mode only):** Describes the relationship between the system clock frequency and the desired frequencies. See [Theory of Operation](#) for details.
- **Frequency per Channel (F_s):** Because of time division multiplexing, the effective system clock to each channel is the real system clock divided by the number of channels.
- **SIN/COS Output Type:** The DDS supports either integer or floating point mode for SIN/COS generation.
- **Parameter Selection:** DDS key parameters can be specified using System Parameters, which are aimed at system architects (frequency domain parameters) or Hardware Parameters, which are aimed at hardware engineers (time-domain parameters). The Phase Generator and SIN/COS LUT are only specified in terms of hardware parameters.

System Parameters

- **Spurious Free Dynamic Range (SFDR):** The targeted purity of the tone produced by the DDS. This sets the output width as well as internal bus widths and various implementation decisions.
- **Frequency Resolution:** Specified in Hz, this specifies the minimum frequency resolution and is used to determine the phase width, as employed by the phase accumulator and its associated phase increment (PINC) and phase offset (POFF) values. Small values give high frequency resolution and require larger accumulators. Larger values reduce hardware resources. Depending upon the choice of noise shaping, the phase width can be increased, and the frequency resolution higher than that specified. For rasterized mode, the frequency resolution is fixed by the system clock, the number of channels, and the modulus selected. See [Theory of Operation](#) for details.
- **Noise Shaping:** This parameter controls whether phase truncation, dithering, or Taylor series correction is used. The options are:
 - **None:** Phase truncation DDS is produced.
 - **Dithering:** Phase dither is used to improve SFDR at the expense of increased noise floor. See [Phase Dithered DDS](#).
 - **Taylor Series Corrected:** Sine/cosine values are interpolated using the otherwise discarded bits from phase truncation. See [Taylor Series Corrected DDS](#).
 - **Auto:** Noise shaping is automatically determined, based on system parameters such as SFDR. The selected noise shaping option is presented in the core summary pages. Auto is only available when the parameter selection is System Parameters.

The availability of particular noise shaping options depends upon the configuration option selected and the parameter selection method. System parameter entry automatically constrains whether a particular noise shaping option is possible. When hardware parameter entry is selected, the options summarized in the following table are made available, and the choice of the noise shaping option then constrains the hardware parameter ranges to those supported by the selected option.

Table 2: Availability of Noise Shaping Options for Hardware Parameters

Setting	DDS Part	Phase Generator Part	SIN/COS LUT Part
None	Available	Available	Available
Dithering	Available		
Taylor	Available		Available
Auto	Available		

Based on the system parameters entered and the noise shaping selected, the minimum phase width and output width are derived by the Vivado IDE in the following way. The phase width can be increased to enable a particular noise shaping option. For example, Taylor series correction requires a minimum phase width of 12 bits.

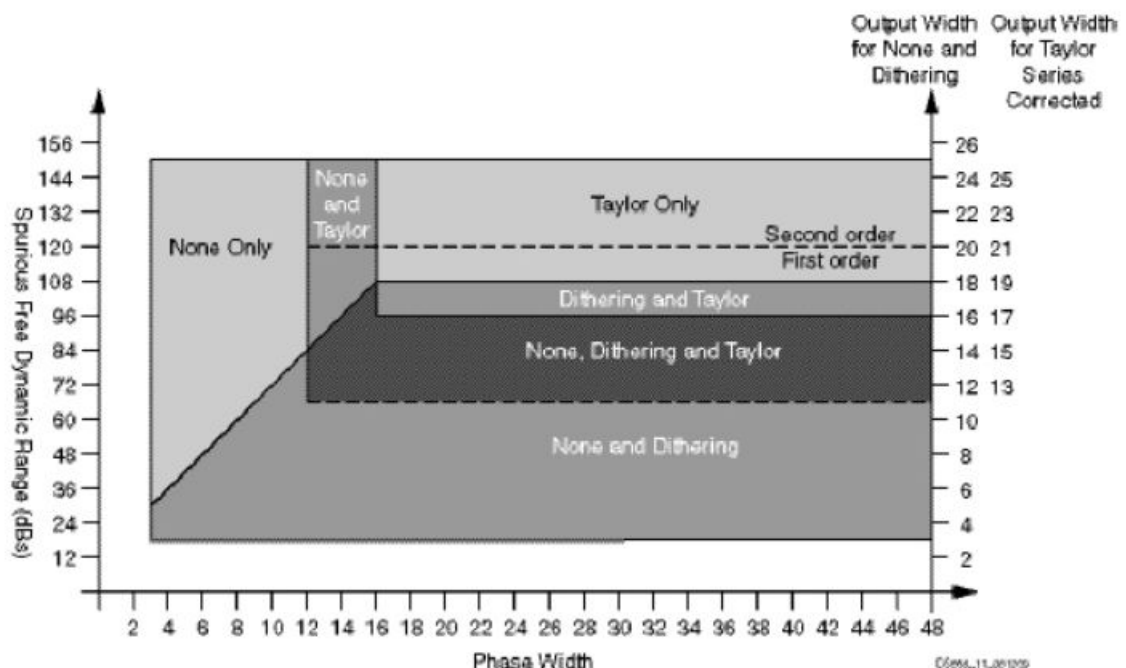
$$\text{Phase Width} = \left\lceil \log_2 \left(\frac{\text{DDS Clock Rate}}{\text{Channels} \times \text{Frequency Resolution}} \right) \right\rceil$$

Table 3: Calculation of Output Width from SFDR and Noise Shaping

Noise Shaping	Output Width (Integer)	Output Width (Float)
None and Dithering	$\text{Output Width} = \left\lceil \frac{\text{SFDR}}{6} \right\rceil$	$\text{Output Width} = 32$
Taylor	$\text{Output Width} = \left\lceil \frac{\text{SFDR}}{6} \right\rceil + 1$	N/A

The following figure shows the regions of SFDR and phase width over which each noise shaping option operates. There are three overlapping regions for None, Phase Dithering and Taylor Series Correction, and deeper levels of shading have been used to show where regions overlap. The darkest region is where all three regions overlap and all three noise shaping options are possible. The lower dashed line signifies that Taylor series correction is only valid for SFDR > 66.0 dB (and not 66.0 dB). Phase width can be increased to maximize the number of noise shaping options for a particular SFDR target.

Figure 26: Noise Shaping Regions



Note: The noise shaping regions only apply to the integer SIN/COS generation.

Hardware Parameters

- **Phase Width:** Sets the width of the PHASE_OUT field within `m_axis_phase_tdata`, the phase field within `s_axis_phase_tdata` when the DDS is configured to be a SIN/COS LUT only, the phase accumulator, associated phase increment and offset registers, and the phase field in `s_axis_config_tdata`. For rasterized mode, the phase width is fixed as the number of bits required to describe the valid input range $[0, \text{Modulus}-1]$, that is $\log_2(\text{Modulus}-1)$ rounded up.
- **Output Width:** Only enabled when the DDS or SIN/COS LUT part is selected (because it is not required by the Phase Generator part), and when the output type is configured to integer. Sets the width of SINE and COSINE fields within `m_axis_data_tdata`. The SFDR that this provides is dependent on the noise shaping option previously selected. The Output Width is set to 32 if floating point is selected as the output type. The equations in the following table can be used to estimate the SFDR that can be achieved:

Table 4: Calculation of SFDR for Given Noise Shaping

Noise Shaping	SFDR
None, Dither	SFDR = Output Width x 6
Taylor	SFDR = (Output Width - 1) x 6

Implementation Tab

- **Phase Increment Programmability:** Selects the means by which the PINC value is set.
 - **Fixed:** PINC is fixed at generation time and cannot be changed at runtime. Fixed requires minimal resource.
 - **Programmable:** PINC value can be changed at runtime using the CONFIG channel. This is recommended when the DDS frequency is to change between modes of operation.
 - **Streaming:** PINC value is taken directly from the input PHASE channel. This is recommended when the PINC value has to change often, or when frequency modulation is required, for example.
- **Resync:** When selected, the `s_axis_phase` channel has a RESYNC field. This bit, when asserted, mutes the accumulated phase for the channel in question. The value of the accumulated phase for that cycle is the value of PINC accompanying the RESYNC assertion, plus the POFF value.
- **Phase Offset Programmability:** Selects the means by which the POFF value is set.
 - **None:** No phase offset facility and the required hardware is not generated. This saves FPGA resources.
 - **Fixed:** PINC is fixed at generation time and cannot be changed at runtime.

- **Programmable:** POFF value can be changed using the CONFIG channel. This is recommended when the DDS phase is to change between modes of operation.
- **Streaming:** POFF value can be changed using the input PHASE channel. This is recommended when the POFF value has to change often, or when phase modulation is required, for example.
- **Output:**
 - **Output_Selection:** The DDS can have a quadrature SINE and COSINE field in the `m_axis_data_tdata` bus, or only one of these two fields. See [Output DATA Channel TDATA Structure](#) for `m_axis_data_tdata` internal structure.
 - **Polarity:** The SINE and COSINE fields of `m_axis_data_tdata` can be inverted. This allows conversion of a DDS used as a transmitter mixer to a receiver mixer, using conjugated outputs; hence both instantiations are identical except for the values of the two selections here.
 - **Negative Sine:** Checking this selection results in the SINE field being negated at runtime.
 - **Negative Cosine:** Checking this selection results in the COSINE field being negated at runtime.
 - **Amplitude Mode:** This selection allows for one of two amplitudes from the DDS.
 - **Full Range:** Aimed at communications applications where the maximum amplitude within the two's complement representation is desired, but the value of amplitude is less important due to the expectation of automatic gain control. The target amplitude for full range mode is $1 - 2^{-(\text{Output Width} - 2)}$ for non-Taylor modes, whereas for Taylor mode the amplitude varies between this value and 1. Here, amplitude is normalized to the output width with the binary point in the first place. An 8-bit output would have a binary amplitude of $10000000 - 10$ giving values between 01111110 and 11111110 corresponding to just less than 1 and just more than -1 respectively.
 - **Unit Circle:** For applications where the value of the DDS output amplitude is important; for example, for FFT twiddle factor generation. When Unit Circle, the DDS output amplitude is half full range (that is, values range from 01000 .. (+0.5). to 110000 .. (-0.5)). As the amplitude is reduced over Full Range by a factor of 2, the SFDR is reduced by 6 dB. Increase SFDR or output width to accommodate this requirement.

Note: Only Unit Circle applies if floating point is selected as the SIN/COS Output Type

Implementation Options

- **Memory Type:** This controls the implementation of the SIN/COS LUT. The Auto setting selects distributed ROM for small cases where the table can be contained in a single layer of memory, and selects block ROM for larger cases (that is, Distributed ROM is selected when $\text{Phase Width} \leq 5$ bits). This selection can be overridden by selecting **Distributed ROM** or **Block ROM** explicitly.

- **Optimization Goal:** In some cases, circuit clock speed can be increased at the expense of extra pipelining registers. This selection controls whether the implementation decisions target highest speed or lowest resource.
- **DSP48 Use:** This controls the implementation of the phase accumulator and following addition stages (for phase offset and/or dither noise addition). When set to Minimal, the phase accumulator and following stages are implemented in FPGA logic. When set to Maximal, all is implemented using DSP slices. In the single channel case, the DSP slice can also provide the register to store programmable phase increment and/or phase offset and thereby save further fabric resources. This is not done if either Phase Increment or Phase Offset is set to Streaming, and only when the Optimization Goal is Area. When this optimization is performed, the initial value of the PINC and/or POFF register must be zero. This is enforced by the Vivado IDE by setting the initial value of PINC and/or POFF to zero and disabling entry.

Detailed Implementation Tab

AXI Channel Options

- **DATA Has TLAST:** Enabled when there is more than one DDS channel (as opposed to AXI channel). Limited options are also available when only the PHASE channel is present. Options are:
 - **Not Required:** In this mode, no TLAST signals are present on the input PHASE channel or the output channels. In multichannel configurations, TLAST on the CONFIG channel is used to denote the last channel to be reconfigured, and is always present, regardless of this setting.
 - **Vector Framing:** A TLAST pulse on the input PHASE channel and output channels denotes the last channel in a cycle of channels (for example, 12th of 12 channels). If the TLAST pulse is not applied at the correct time to match the channel state of the core, an event is flagged on the `event_s_phase_tlast_missing` or `event_s_phase_tlast_unexpected` event outputs.
 - **Packet Framing:** A TLAST pulse is conveyed from the input PHASE channel to the output channels with the same latency as TDATA. TLAST in this configuration can be used to trigger a reconfiguration. This mode is intended as a service to ease system design for cases where signals must accompany the datastream, but which have no application in the DDS.
 - **Config Triggered:** This option causes the core to generate an output TLAST on the last TDM channel before a new configuration is applied to the core. Subsequent output samples are generated using the new core configuration. This mode is only available when the CONFIG channel is present.
- **Output TREADY:** When selected, the output channels have a TREADY and hence support the AXI4handshake protocol with inherent back pressure. If there is an input PHASE channel, the presence of its TREADY is also determined by this control, so that the datapath from input PHASE channel to output channels as a whole supports back pressure or not.

- **TUSER options:** The core supports two distinct uses of the TUSER field; to denote the time-division-multiplex channel index or as conduit to pass a user field (auxiliary data associated with TDATA) from input PHASE channel to output channels. These choices are independent for the input PHASE channel. However, because the selection of a user field implies the desire to convey the TUSER field from input to output, the selection of a user field on the input PHASE channel forces a user field to be present in each of the output channel TUSER ports. Options for the input PHASE channel are shown below. Options for the each output channel are constrained by the input PHASE channel choice, but are otherwise independent.
- **Not required:** Neither of the above uses is required; the channel in question does not have a TUSER field.
- **Chan ID field:** In this mode, the TUSER field identifies the time-division-multiplexed channel for the transfer. For the input PHASE channel, this gives the user a mechanism to synchronize to the internal DDS channel state. If the applied channel ID does not match the internal state of the core, an event is flagged on the `event_s_phase_chanid_incorrect` output.
- **User Field:** In this mode, the core ignores the content of the TUSER field, but passes it unaltered from input PHASE channel to the output channels.
- **User and Chan ID field:** In this mode the TUSER field has both a user field and a channel ID field, with the channel ID field in the least significant bits. The minimal number of bits required to describe the channel determines the width of the channel ID field; for example, seven channels require three bits.
- **User field width:** This field determines the width of the bit field that is conveyed from input to output unaltered by the DDS. It does not include the width of the channel ID field, if it is present.
- **Output Form:** The output form is indirectly controlled by the SIN/COS Output Type parameter. When the output type is set to an integer, the two's complement and sign and magnitude options are available to the Output Form. When the output type is set to floating point, only the single precision float is available. In general, the output of SINE and COSINE is in two's complement form. However, when quadrant symmetry is used, the output form can be changed to sign and magnitude. Selecting sign and magnitude removes the inverters following the SIN/COS LUT. This cuts resource use relative to two's complement. The single precision float behaves similarly to sign and magnitude. It is intended for use with a mixer stage where the sign information can be handled separate to the magnitude by changing the sign of the components in a complex multiplication. A mixer (complex multiplier with inputs $a + jb$ and $c + jd$) can be expressed as:

$$\text{Real} = ac - bd$$

$$\text{Imaginary} = ad + bc$$

In this case, a and b from the DDS are signed, two's complement. However, if a and b are expressed as sign and magnitude, the sign from each of a and b can be used to modify the sign of each of the terms in the mixer equation. For instance, if a alone is negative, the mixer equation becomes:

Real = $-ac - bd$

Imaginary = $-ad + bc$

If the mixer stage is implemented using DSP slices, this sign manipulation can be implemented by changing the DSP48 slice opmode signal.

- **Synchronization Mode:** This selection deals with the timing of reconfiguration when both CONFIG and PHASE channels are present. The configuration channel takes configuration data asynchronously to the phase of the channel counter and stores the reconfiguration data in a buffer. This selection determines when that new configuration data takes effect on the datapath:
 - **On Vector:** In this mode, the reconfiguration data is applied when the channel counter rolls over to start a new cycle of time-division-multiplexed channels.
 - **On Packet:** In this mode, available when TLAST is set to packet framing, a TLAST on the input PHASE channel triggers the reconfiguration. This mode is targeted at cases where each set of configuration data is to be associated with the packets implied by the input TLAST indicator.

Latency Options

Select whether latency should be configured automatically by the Vivado IDE or manually:

- **Auto:** Causes the DDS to be pipelined for optimal performance (taking into account the optimization goal).
- **Configurable:** Where optimal performance is beyond requirements, latency can be set to configurable and a smaller value of latency selected. This reduces the number of pipeline stages and generally results in resource savings. A minimum value of latency is imposed, where a cycle of latency arises from each of the following sources:
 - Streaming phase increment
 - Block ROM within SIN/COS LUT (this can be avoided if you select **Distributed ROM**).
 - Block ROM within second order Taylor series correction (used for SFDR above 120 dB).

Note: When TREADY is selected the AXI interfaces buffer data as a FIFO. This buffering results in non-deterministic latency. However, this action can only increase in latency. In this case, the minimum latency possible is six cycles plus the minimum latency described above.

Optional Pins

Certain inputs and outputs can be disabled to save resources.

- **Has Phase Out:** When checked the core has the output PHASE channel.
- **ACLKEN:** When checked the core has an `aclken` (active-High clock enable) port.

- **ARESETn:** When checked the core has an `aresetn` (active-Low synchronous reset) port.

Parameter Entry Pages

The following pages appear for entry of parameters when either Phase Increment or Phase Offset are either Fixed or Programmable. If Programmable, the initial value of the register is specified through the Parameter Entry Pages. If an DSP slice register is used, the initial value of phase increment and/or offset is assumed to be zero.

- **System Parameters:**
 - **Output Frequencies:** This page appears when Parameter Selection is set to System Parameters and Phase Increment Programmability is Fixed or Programmable. For each channel, an independent frequency (MHz) can be entered into the table. The allowable range is displayed as 0 to F_s (where F_s is the frequency per channel). Values from $F_s/2$ to F_s alias to $-F_s/2$ to 0 respectively, so can be used to input negative frequencies.
 - **Phase Offset Angles:** This page appears when Parameter Selection is set to System Parameters and Phase Offset is set to Fixed or Programmable. This table allows the phase offset to be specified for each channel as a fraction of a cycle. The valid range is -1.0 to 1.0 for standard mode. For rasterized mode, the valid range is 0 to 1.0. For example, enter 0.5 for 180° (that is, π radians). The range for standard mode is greater than a single cycle, but is allowed because negative values map to equivalent positive values.
- **Hardware Parameters:**
 - **Phase Angle Increment Values:** This page appears when Parameter Selection is set to Hardware Parameters and Phase Increment Programmability is Fixed or Programmable. Values must be entered in binary. The range is 0 to the Modulus value minus 1. For standard mode, the Modulus value is $2^{\text{PhaseWidth}}$. For rasterized mode, the modulus is a value selected earlier in the Vivado IDE. The angle in radians is the input number divided by the Modulus and multiplied by 2π . Entries are extended to phase width bits by zero padding to the left.
 - **Phase Offset Values:** This page appears when Parameter Selection is set to Hardware Parameters and Phase Offset is set to Fixed or Programmable. Values must be entered in binary. The range for standard mode is 0 to the weight of the accumulator, that is, $2^{\text{PhaseWidth}} - 1$, which corresponds to a single cycle. For rasterized mode, the valid range is 0 to $1.0 - 1/\text{Modulus}$. The angle in radians can be obtained by converting the unsigned fractional number to decimal and multiplying by 2π . Entries are extended to phase width bits by zero padding to the left.

Summary (Two Pages)

The final two tabs provide feedback fields.

- **Summary (Page 1):** This page presents the resolved values of the selected part. For instance, these fields indicate the result of automatic memory type and latency allocation. They also indicate the expected SFDR and frequency resolution for the DDS when hardware parameters are used for input, or vice versa. There are also resource estimates (DSP slices and 18 Kb block RAM primitives).
- **Summary (Page 2):** This page is only presented when Phase Increment and/or Phase Offset are fixed or programmable, and provides a summary of the hexadecimal values used to obtain a particular frequency or phase offset. The actual value of frequency and phase (the latter as a fraction of a cycle) is also given as a floating-point number.

User Parameters

The following table shows the relationship between the fields in the AMD Vivado™ IDE and the user parameters (which can be viewed in the Tcl Console).

Table 5: User Parameters

Vivado IDE Parameter/Value	User Parameter/Value	Default Value
Configuration Options	partspresent	Phase_Generator_and_SIN_COS_LUT
System Clock (MHz)	dds_clock_rate	100
Number of Channels	channels	1
Mode of Operation	mode_of_operation	Standard
Modulus	modulus	9
Parameter Selection	parameter_entry	System_Parameters
Spurious Free Dynamic Range	spurious_free_dynamic_range	45
Frequency Resolution	frequency_resolution	0.4
Noise Shaping	noise_shaping	Auto
Phase Width	phase_width	16
Output Width	output_width	12
Phase Increment Programmability	phase_increment	Fixed
Resync	resync	False
Phase Offset Programmability	phase_offset	None
Output Selection	output_selection	Sine_and_Cosine
Negative Sine	negative_sine	False
Negative Cosine	negative_cosine	False
Amplitude Mode	amplitude_mode	Full_range
Memory Type	memory_type	Auto
Optimization Goal	optimization_goal	Auto
DSP48 Use	dsp48_use	Minimal
Has Phase Out	has_phase_out	True
DATA has TLAST	data_has_tlast	Not_required
Output TREADY	has_tready	False

Table 5: User Parameters (cont'd)

Vivado IDE Parameter/Value	User Parameter/Value	Default Value
TUSER Options: Input	s_phase_has_tuser	Not_required
TUSER Options: User Field Width	s_phase_tuser_width	1
TUSER Options: DATA Output	m_data_has_tuser	Not_required
TUSER Options: PHASE Output	m_phase_has_tuser	Not_required
Synchronization Mode	s_config_sync_mode	On_Vector
SIN/COS Output Type	output_type	Integer
Output Form	output_form	Twos_complement
Latency Configuration	latency_configuration	Auto
Latency	latency	1
ARESETn	has_aresetn	False
ACLKEN	has_aclken	false

Output Generation

For details, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

System Generator for DSP

The DDS Compiler core is in the AMD adaptive computing blockset in the DSP section of the System Generator. The block is called the DDS Compiler.

The System Generator for DSP GUI offers the same parameters as the Vivado design tools GUI. There is, however, a minor difference for the hardware parameters. In the Vivado design tools GUI, the hardware parameters are hidden when you select **System Parameters**. In the System Generator for DSP GUI, the hardware parameters are disabled. Likewise, the system parameters are disabled when you select **Hardware Parameters**.

For more information on System Generator for DSP, see the *System Generator for DSP User Guide* ([UG640](#)).

Constraining the Core/Subsystem

Required Constraints

This section is not applicable for this IP core.

Device, Package, and Speed Grade Selections

This section is not applicable for this IP core.

Clock Frequencies

This section is not applicable for this IP core.

Clock Management

This section is not applicable for this IP core.

Clock Placement

This section is not applicable for this IP core.

Banking

This section is not applicable for this IP core.

Transceiver Placement

This section is not applicable for this IP core.

I/O Standard and Placement

This section is not applicable for this IP core.

Simulation

For comprehensive information about AMD Vivado™ simulation components, as well as information about using supported third-party tools, see the *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#)).

Synthesis and Implementation

For details about synthesis and implementation, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

C Model

The DDS Compiler core bit-accurate C model is a self-contained, linkable, shared library that models the functionality of this core with finite precision arithmetic. This model provides a bit-accurate representation of the various modes of the DDS Compiler core, and is suitable for inclusion in a larger framework for system-level simulation or core-specific verification.

The C model is an optional output of the AMD Vivado™ Design Suite (see the Vivado documentation to set up the C model output).

Features

- Bit accurate with the DDS Compiler core
- Available for 64-bit Linux platforms
- Available for 64-bit Windows platforms
- Supports all features of the DDS Compiler core with the exception of those affecting timing or AXI4-Stream configuration
- Designed for integration into a larger system model
- Example C code showing how to use the C model functions

Overview

The model consists of a set of C functions that reside in a shared library. Example C code is provided to demonstrate how these functions form the interface to the C model. Full details of this interface are given in [C Model Interface](#).

The model is bit accurate but not cycle accurate; it performs exactly the same operations as the core. It does not, however, model the core latency, its interface signals, or the TUSER feature.

Unpacking and Model Contents

There are separate ZIP files containing all the files necessary for use. Each ZIP file contains:

- C model shared library
- C model header file
- Example code showing how to call the C model

The following tables list the contents of each ZIP file.

Table 6: C Model Zip File Contents: Linux

File	Description
dds_compiler_v6_0_bitacc_cmodel.h	Header file which defined the C model API
libIp_dds_compiler_v6_0_bitacc_cmodel.so	Model shared object library
Run_bitacc_cmodel.c	Example program for calling the C model.
dds_compiler_v6_0_bitacc_mex.cpp	MATLAB MEX function source
make_dds_compiler_v6_0_mex.m	MATLAB MEX function compilation script
run_dds_compiler_v6_0_mex.m	MATLAB MEX function example script
@dds_compiler_v6_0_bitacc	MATLAB MEX function class directory

Table 7: C Model Zip File Contents: Windows

File	Description
dds_compiler_v6_0_bitacc_cmodel.h	Header file which defined the C model API
libIp_dds_compiler_v6_0_bitacc_cmodel.dll	Model dynamically linked library
libIp_dds_compiler_v6_0_bitacc_cmodel.lib	Model LIB file for compiling
run_bitacc_cmodel.c	Example program for calling the C model
dds_compiler_v6_0_bitacc_mex.cpp	MATLAB MEX function source;
make_dds_compiler_v6_0_mex.m	MATLAB MEX function compilation script;
run_dds_compiler_v6_0_mex.m	MATLAB MEX function example script;
@dds_compiler_v6_0_bitacc	MATLAB MEX function class directory

Installation

Linux

- Unpack the contents of the zip file.

- Ensure that the directory where the `libIP-dds_compiler_v6_0_bitacc_cmodel.so` resides is included in the path of the environment variable `LD_LIBRARY_PATH`.

Windows

- Unpack the contents of the zip file.
- Ensure that the directory where the `libIP-dds_compiler_v6_0_bitacc_cmodel.dll` resides is:
 - Included in the path of the environment variable `PATH`, or
 - In the directory in which the executable that calls the C model is run.

C Model Interface

An example file, `run_bitacc_cmodel.c`, is included. This demonstrates how to call the C model. See this file for examples of using the interface described in this section.

The application programming interface (API) of the C model is defined in the header file `dds_compiler_v6_0_bitacc_cmodel.h`. The interface consists of data structures and functions as described in the following sections.

Data Types

The following table shows the types defined for the DDS Compiler C model.

Table 8: C Model Data Types

Name	Type	Description
<code>xip_dds_v6_0_data</code>	Double	Used for phase input/output and for SIN/COS output.
<code>xip_uint</code>	Unsigned Int	Used for configuration parameter of integer or Boolean type. For Boolean: 0 = False 1 = True
<code>xip_dds_v6_0_config</code>	Struct	Structure of configuration parameters.
<code>xip_dds_v6_0_config_pkt</code>	Struct	Data structure of phase increment (PINC) and/or phase offset (POFF) values input to the core at runtime. Equivalent to data on the <code>S_AXIS_CONFIG</code> channel of the core.
<code>xip_dds_v6_0_status</code>	Int	Error code return from many C model functions. 0 indicates success. Any other value indicates failure.
<code>xip_status</code>	Int	Same as <code>xip_dds_v6_0_status</code> , but used for functions which are not core-specific.
<code>xip_real</code>	Double	

Table 8: C Model Data Types (cont'd)

Name	Type	Description
xip_array_real	Struct	Structure used to hold data for input to DDS (on S_AXIS_PHASE channel) or output from DDS.
xip_dds_v6_0_config	Struct	The configuration of the core itself. The members of this structure are listed in the dds_compiler_v6_0_bitacc_cmodel file. The names closely match the same names in XCO/XCI files. The dds_compiler_v6_0_bitacc_cmodel file also contains #defined values for all. All member variables beginning with "res" are for internal use and need not be set.
xip_dds_v6_0	Struct	Type defined which C (not C++) can use as a handle (pointer) to a C++ object – the C model itself.

`xip_dds_v6_0_data` is of type double because this has sufficient precision to describe even the largest input and output vectors (48 bits) without the complications of long which can vary from platform to platform or compiler to compiler.

`xip_array_real` is a structure with the following members:

- **data:** A pointer to the array of data values.
- **data_size:** Of type `size_t`, which describes the total size of the data array.
- **data_capacity:** Also of type `size_t`, which describes how much of the array is currently populated.
- **dim:** A pointer to a `size_t` array of values that indicate the size of each dimension.
- **dim_size (size_t):** Indicates the number of dimensions of the data array.
- **dim_capacity:** Indicates how much of the dimension array is currently populated.
- **owner, unsigned int:** This is provided as a handle for when the data structure is intended to be passed from one core to another, but is not used by any of the DDS C model functions.

Functions

There are several accessible C model functions.

Information Functions

The following table lists the information functions.

Table 9: Information Functions

Name	Return	Arguments	Description
xip_dds_get_version	Const char*	Void	Return the DDS Compiler C model version as a null terminated string. For v6.0, this is "6.0".
xip_dds_v6_0_get_default_config	xip_dds_v6_0_status	xip_dds_v6_0_config*	Populates the contents of structure pointed to by the input argument with the values of a default configuration.

Initialization Functions

The functions to create, configure, and destroy the C model and associated data structures are listed in the following table.

Table 10: Initialization Functions

Name	Return	Arguments	Description
xip_dds_v6_0_create	Pointer to structure holding configuration of C model object	Pointer to structure holding configuration	Creates new C model object and returns pointer to config structure (which is pointer to C model itself).
xip_dds_v6_0_destroy	xip_dds_v6_0_status	Pointer to xip_dds_v6_0 (C model itself)	Deallocates memory owned by the C model and destroys the C model itself.
xip_dds_v6_0_get_config	xip_dds_v6_0_status	Pointer to C model, pointer to configuration structure	Copies the contents of the configuration of the C model indicated to the designated configuration structure.
xip_array_create	Pointer to created data structure	None	Allocates memory for the structure itself, not the array members within it.
xip_array_reserve_data	xip_status	Pointer to data structure, maximum number of elements in data array	(Re)allocates enough memory for the maximum size. An error is returned if structure data_capacity is greater than the space allocated.
xip_array_reserve_dim	xip_status	Pointer to data structure, maximum number of dimensions	Allocates a small array which is to contain the size of each dimension of the data array. For example, 100 samples x 4 channels x 3 fields.
xip_array_destroy	xip_status	Pointer to data structure	Frees up the memory allocated for the data array, the dimension array, and the data structure itself.

Table 10: Initialization Functions (cont'd)

Name	Return	Arguments	Description
xip_dds_v6_0_alloc_config_pkt	xip_dds_v6_0_status	Pointer to xip_dds_v6_0_config_pkt, xip_uints for number of PINC values and number of POFF values	Allocates memory for arrays within config_pkg, but not config_pkt itself.
xip_dds_v6_0_free_config_pkt	xip_dds_v6_0_status	Pointer to xip_dds_v6_0_config_pkt	Deallocates memory for arrays within config_pkt.

Execution Functions

The runtime functions of the C model are described in the following figure.

Table 11: Execution Functions

Name	Return	Arguments	Description
xip_dds_v6_0_reset	xip_dds_v6_0_status	Pointer to C model (xip_dds_v6_0)	Resets the internal state of the core to power-on state.
xip_dds_v6_0_config_do	xip_dds_v6_0_status	Pointer to C model, pointer to config_pkt to input to C model	Writes config_pkt to C model internal storage. This does not prompt execution of the C model, but the config data is picked up on the next xip_dds_v6_0_data_do call
xip_dds_v6_0_data_do	xip_dds_v6_0_status	Pointer to C model, Pointer to input data structure, Pointer to output data structure, Number of samples, number of channels, number of fields input and number of fields output	The function which prompts execution of the C model. The number of samples, channels and fields must match the size of the array passed or an error is returned.
xip_array_real_set_data	xip_status	Pointer to array structure, the value to be written, the sample, channel and field to be written to	Used to populate the input data structure.
xip_array_real_get_data	xip_status	Pointer to the array structure, pointer of real type (returned value), sample, channel and field to be read	Used to read the output (or input) data structure.

Data Format

The DDS Compiler sine and cosine outputs are generated as unsigned integer or single precision float values that are cast to doubles on the interface. To restore the sign for printing to `stdout` or use outside the C model, you must either cast back to integer and scale accordingly for the integer output type, or cast back to float for the float output type. Because both sine and cosine can be a maximum of 26 bits wide (integer), casting to an int is sufficient. To scale the data, it is recommended that the data is shifted up by the difference between the width of an int (usually 32 bits) and the `config.Output_Width` C model parameter to set the sign bit correctly, then scaled down by the same value, maintaining the sign.

For example, for a 12-bit output, the scaling value is $(32 - 12) = 20$, and the correctly-signed sine output can be printed using the following code:

```
std::cout << "sine " << ((int)sine_output << 20) >> 20) << std::endl;
```

See the C model smoke test, `run_bitacc_cmodel.c`, for a generalized example.

The phase output from the C model can be up to 48 bits wide, and can be either signed or unsigned, depending on the configuration. See [Output PHASE Channel](#) for more details. In the general case, for a signed phase output, cast the double to a 64-bit integer type (for example, `int64_t`), and for an unsigned phase output, cast the double to an unsigned 64-bit integer type (for example, `uint64_t`).

Compiling

Compilation of user code requires access to the `dds_compiler_v6_0_bitacc_cmodel.h` header file. The header file should be copied to a location where it is available to the compiler. Depending on the location chosen, the include search path of the compiler might need to be modified.

When compiling on Windows, the symbol `NT` must be defined, either by a compiler option, or in user source code before the

`dds_compiler_v6_0_bitacc_cmodel_v6_0_bitacc_cmodel.h` header file is included.

Linking

To use the C model, the user executable must be linked against the correct libraries for the target platform.

Linux

The executable must be linked against the `libIp_dds_compiler_v6_0_bitacc_cmodel.so` library.

Using GCC, linking is typically achieved by adding the following command line options:

```
-L. -Wl, -rpath,. -lIp_dds_compiler_v6_0_bitacc_cmodel
```

This assumes the object library is in the current directory. If this is not the case, the `-L.` option should be changed to specify the library search path to use.

Using GCC, the provided example program `run_bitacc_cmodel.c` can be compiled and linked using the following command:

```
gcc -x c++ -I. -L. -lIp_dds_compiler_v6_0_bitacc_cmodel -Wl, -rpath,. -o  
run_bitacc_cmodel run_bitacc_cmodel.c
```

Windows

The executable must be linked against the `libIp_dds_compiler_v6_0_bitacc_cmodel.dll` dynamic link library.

Depending on the compiler, the `libIp_dds_compiler_v6_0_bitacc_cmodel.lib` import library might also be required.

To link to an import library using Microsoft Visual Studio, add the library to the Additional Dependencies entry under the Linker section of Project Properties.

Example

The `run_bitacc_cmodel.c` file contains example code to show basic operation of the C model. For example, the file shows declaration for the model, configuration of the model, allocation of required input and output data structures, execution of the model, resetting of the core, deallocation of the data structures, and destruction of the model.

The example file does not verify the output data for correctness, but allows a debug mode in which output data can be printed to a standard output channel (stdout).

MATLAB Interface

A MEX function and MATLAB® software class is provided to simplify the integration with MATLAB. The MEX function provides a low-level wrapper around the underlying C model, while the class file provides a convenient interface to the MEX function.

Compiling

Source code for a MATLAB MEX function is provided. This can be compiled within MATLAB by changing to the directory which contains the code and running the `make_dds_compiler_v6_0_bitacc_mex.m` script.

Installation

To use the MEX function the compiled MEX function must be present on the MATLAB search path. This can be achieved by either of the following:

1. Adding the directory where the compiled MEX function is located to the MATLAB search path (see the MATLAB `addpath` function), or
2. Copying the files to a location already on the MATLAB search path.

As with all uses of the C model, the correct C model libraries also need to be present on the platform library search path (that is, `PATH` or `LD_LIBRARY_PATH`).

MATLAB Class Interface

The `@dds_compiler_v6_0_bitacc` class handles the create/destroy semantics on the C model. The class provides objects for each of the data and control structures defined for the C model and described in [Data Types](#). MATLAB arrays are used for the mapping of types as in the following table.

Table 12: C Model Type Mapping

C Model Type	MATLAB Type
xip_real	double

The class provides the following methods.

Constructor

```
[model] = dds_compiler_v6_0_bitacc  
[model]=dds_compiler_v6_0_bitacc(config)  
[model]=dds_compiler_v6_0_bitacc(field, value [, field,value]*)
```

* indicates an optional parameter.

The first version of the function call constructs a model object using the default configuration. The second version constructs a model object from a structure that specified the configuration parameter values to use. The third version is the same as the second, but allows the configuration to be specified as a series of (parameter name, value) pairs rather than a single structure. The names and valid values of configuration parameters are identical to those described for the C model in [Data Types](#).

The MATLAB configuration structure can contain an additional element, `PersistentMemory`. When the element is set to `TRUE` the internal data memory state of the model is retained following a call to the run function. Otherwise, the model is reset after the data is returned. `PersistentMemory` is set to `FALSE` by default.

Get Version

```
[version] = get_version(model)
```

This method returns the version string of the C model library used.

Get Configuration

```
[config] = get_configuration(model)
```

This method returns the current parameters structure of a model object. If the model object is empty the method returns the default configuration. If the model object has been created, the method returns the configuration parameters that were used to create it.

Reset

```
[model] = reset(model)
```

This function resets the model.

Config_do

```
config_do(model, config_pkt)
```

This method applies new values of PINC and/or POFF to the model, depending on whether Phase Increment and Phase Offset have been configured as programmable. This function only applies if at least one of Phase Increment or Phase Offset have been configured as programmable.

Run

```
[data_out]=run(model, data_samples)  
[data_out]=run(model, data_samples, data_in)
```

Each of these methods causes the model to be run for the number of cycles specified by the `data_samples` value.

The first form applies if the model has been configured with neither Phase Increment nor Phase Offset configured as Streaming.

The second form applies an array of values for PINC and/or POFF. Each element of `data_in` should have a value for PINC (if Phase Increment has been configured as Streaming) and/or POFF (if Phase Offset has been configured as Streaming). The number of elements in `data_in` should match `data_samples`.

Data out is an array of output samples. Each element contains a phase, sin and cosine value output depending on the configuration of the model. The number of output elements is the `data_samples` value.

Destroy

```
destroy(model)
```

This method destroys all memory associated with the model.

Example

The `run_dds_compiler_v6_0_bitacc_mex.m` file contains a MATLAB script with several examples of differently configured DDS models showing how to configure and run each. Each example configuration and run produces a plot of the output values.

To run the sample script:

1. Compile the MEX function with the `make_dds_compiler_v6_0_bitacc_mex.m` script (see [Compiling](#)).
2. Install the MEX function (see [Installation](#)).
3. Execute the `run_dds_compiler_v6_0_bitacc_mex.m` script.

Test Bench

This chapter contains information about the demonstration test bench provided in the AMD Vivado™ Design Suite.

Demonstration Test Bench

When the core is generated using the Vivado Design Suite, a demonstration test bench is created. This is a simple VHDL test bench that exercises the core.

The demonstration test bench source code is one VHDL file, `demo_tb/tb_<component_name>.vhd`, located in the Vivado output directory. The source code is comprehensively commented.

Using the Demonstration Test Bench

The demonstration test bench instantiates the generated DDS Compiler core. Compile the netlist and the demonstration test bench into the work library (see your simulator documentation for more information on how to do this), then simulate the demonstration test bench. View the test bench signals in your simulator waveform viewer to see the operations of the test bench.

Demonstration Test Bench in Detail

The demonstration test bench performs the following tasks:

- Instantiate the core
- Generate a clock signal
- Drive the core input signals to demonstrate core features
- Check that the core output signals obey AXI protocol rules (to keep the test bench simple, data values are not checked)
- Provide signals showing the separate fields of AXI TDATA and TUSER signals

The operations performed by the demonstration test bench are appropriate for the configuration of the generated core:

- If phase increment and offset are fixed:
 - Run to produce sine / cosine / phase outputs
- If phase increment and/or offset are programmable, and neither is streaming:
 - Program an initial configuration
 - Run to produce sine / cosine / phase outputs
 - Program a different configuration
 - Run again to produce sine / cosine / phase outputs
- If one of phase increment or offset are streaming and the other is fixed:
 - Stream in constant phase increment or offset to produce sine / cosine / phase outputs
 - If phase offset is streaming, stream in incrementing phase offset to produce higher frequency sine / cosine / phase outputs
- If one of phase increment or offset are streaming, and the other is programmable:
 - Program an initial configuration
 - Stream in constant phase increment or offset to produce sine / cosine / phase outputs
 - If phase offset is streaming, stream in incrementing phase offset to produce higher frequency sine / cosine / phase outputs
 - Continue streaming in phase increment or phase offset, and simultaneously program a different configuration
- If phase increment and offset are both streaming:
 - Stream in constant phase increment and zero phase offset to produce sine / cosine / phase outputs
 - Stream in zero phase increment and incrementing phase offset to produce sine / cosine / phase outputs
- For SIN/COS LUT only:
 - Stream in incrementing phase to produce sine / cosine outputs
- For all configurations:
 - Demonstrate back pressure by deasserting TREADY of master channels (if TREADY is present)
 - Demonstrate use of clock enable (if present)
 - Demonstrate use of reset (if present)

Customizing the Demonstration Test Bench

It is possible to modify the demonstration test bench to drive the core inputs with different data or to perform different operations.

All operations performed by the demonstration test bench to drive the core inputs are done in the `stimuli` process. This process is comprehensively commented, to explain clearly what is being done. New operations, potentially with different input data, can be added by copying and modifying sections of this process.

The clock frequency of the core can be modified by changing the `CLOCK_PERIOD` constant.

Upgrading

This appendix contains information about migrating a design from the Design Suite to the AMD Vivado™ Design Suite, and for upgrading to a more recent version of the IP core. For customers upgrading in the Vivado Design Suite, important details (where applicable) about any port changes and other impact to user logic are included. This appendix describes migrating from older versions of the IP to the current IP release.

Migrating to the Vivado Design Suite

For information about migrating to the Vivado Design Suite, see the *ISE to Vivado Design Suite Migration Guide* ([UG911](#)).

Upgrading in the Vivado Design Suite

Parameter Changes

There are no changes to parameters from v5.0 to v6.0. However, there are some new parameters in v6.0 that default to values so that v6.0 is backwards compatible with v5.0. The following table lists the parameter changes from v4.0 to v6.0.

Table 13: Parameter Changes

Version 4.0	Version 6.0	Notes
PartPresent	PartPresent	Unchanged
DDS_Clock_Rate	DDS_Clock_Rate	Unchanged
Channels	Channels	Unchanged
	Mode_of_Operation	New to v6.0
	Modulus	New to v6.0
Parameter_Entry	Parameter_Entry	Unchanged
Spurious_Free_Dynamic_Range	Spurious_Free_Dynamic_Range	Unchanged
Frequency_Resolution	Frequency_Resolution	Unchanged
Noise_Shaping	Noise_Shaping	Unchanged

Table 13: Parameter Changes (cont'd)

Version 4.0	Version 6.0	Notes
Phase_Width	Phase_Width	Unchanged
Output_Width	Output_Width	Unchanged
Phase_Increment	Phase_Increment	Unchanged
	Resync	New to v6.0
Phase_Offset	Phase_Offset	Unchanged
Output_Selection	Output_Selection	Unchanged
Negative_Sine	Negative_Sine	Unchanged
Negative_Cosine	Negative_Cosine	Unchanged
Amplitude_Mode	Amplitude_Mode	Unchanged
Memory_Type	Memory_Type	Unchanged
Optimization_Goal	Optimization_Goal	Unchanged
DSP48_Use	DSP48_Use	Unchanged
Latency_Configuration	Latency_Configuration	Unchanged
Latency	Latency	Unchanged
Has_Phase_Out	Has_Phase_Out	Unchanged
SCLR_pin	Has_ARESETn	Name change only. The signal itself, aresetn, is active-Low and must be driven Low for at least two cycles.
Clock_Enable	Has_ACLKEN	Name change only.
RFD		Deprecated. Closest equivalent is Has_TREADY.
RDY		Deprecated. Equivalent to AXI4-Stream TVALID, which is not optional.
Channel_Pin		Deprecated. The presence of a channel indication field is now specified by M_DATA_Has_TUSER and M_PHASE_Has_TUSER.
Output_Frequency(1 to 16)	Output_Frequency(1 to 16)	Unchanged
PINC(1 to 16)	PINC(1 to 16)	Unchanged
Phase_Offset_Angles(1 to 16)	Phase_Offset_Angles(1 to 16)	Unchanged
POFF(1 to 16)	POFF(1 to 16)	Unchanged
POR_mode	POR_mode	Unchanged
	DATA_Has_TLAST	New to v5.0
	S_PHASE_Has_TUSER	New to v5.0
	S_PHASE_TUSER_Width	New to v5.0
	Has_TREADY	New to v5.0
	M_DATA_Has_TUSER	New to v5.0
	M_PHASE_Has_TUSER	New to v5.0
	S_CONFIG_Sync_Mode	New to v5.0
	Output_Type	New to v6.0
	OUTPUT_FORM	New to v6.0

Port Changes

Version 6.0 includes optional new ports or outputs that might not be relevant to those present in v5.0. These new optional ports default to a state that are ignored by v5.0 designs. The following table details the changes to port naming, additional or deprecated ports and polarity changes from v4.0 to v6.0.

Table 14: Port Changes from Version 4.0 to Version 6.0

Version 4.0	Version 6.0	Notes
CLK	aclk	Rename only.
CE	aclken	Rename only.
SCLR	aresetn	Rename, change of sense (now active-Low), requirement to drive reset Low for a minimum of two cycles.
ADDR	Deprecated	Replaced by s_axis_config_t* (CONFIG channel).
REG_SELECT		
WE		
DATA		
PINC_IN	Deprecated	Replaced by s_axis_phase_t* (input PHASE channel).
POFF_IN		
PHASE_IN	Deprecated	Replaced by s_axis_phase_t* (input PHASE channel).
RDY	Deprecated	Nearest equivalent is m_axis_data_tvalid.
RFD	Deprecated	Nearest equivalent is s_axis_phase_tready.
CHANNEL	Deprecated	Channel ID can be carried as a subfield of m_axis_phase_tuser or m_axis_data_tuser.
COSINE	Deprecated	Both these fields are now subfields of m_axis_data_tdata. See Output DATA Channel TDATA Structure .
SINE		
PHASE_OUT(N-1:0) ¹	m_axis_phase_tdata(byte(N)-1: 0) ¹	PHASE_OUT is now the payload of m_axis_phase_tdata.
-	s_axis_config_tvalid	TVALID (AXI4-Stream channel handshake signal) for each channel.
-	s_axis_phase_tvalid	
-	m_axis_phase_tvalid	
-	m_axis_data_tvalid	
-	s_axis_config_tready	TREADY (AXI4-Stream channel handshake signal) for each channel.
-	s_axis_phase_tready	
-	m_axis_phase_tready	
-	m_axis_data_tready	

Table 14: Port Changes from Version 4.0 to Version 6.0 (cont'd)

Version 4.0	Version 6.0	Notes
-	s_axis_config_tlast	TLAST (AXI4-Stream packet signal indicating the last transfer of a data structure) for each channel. See the TLAST user section for each channel in AXI4-Stream Considerations .
-	s_axis_phase_tlast	
-	m_axis_phase_tlast	
-	m_axis_data_tlast	
-	s_axis_phase_tuser (E-1:0) ¹	TUSER (AXI4-Stream ancillary field for application-specific information) for each channel. See the TUSER packing section for each channel in AXI4-Stream Considerations .
-	m_axis_phase_tuser (F-1:0) ¹	
-	m_axis_data_tuser (G-1:0) ¹	
-	event_s_phase_tlast_missing	Asserted on the last transaction of an incoming vector if s_axis_phase_tlast is not seen asserted.
-	event_s_phase_tlast_unexpected	Asserted on every transaction where s_axis_phase_tlast is unexpectedly seen asserted.
-	event_s_phase_chained_incorrect	Asserted on every transaction where the s_axis_phase_tuser channel ID field does not match the value expected by the core.
-	event_s_config_tlast_missing	Asserted on the last transaction of an incoming vector if s_axis_config_tlast is not seen asserted.
-	events_s_config_tlast_unexpected	Asserted on every transaction where s_axis_config_tlast is unexpectedly seen asserted.
-	event_pinc_invalid	New to v6.0. Indicates an invalid value of PINC when in rasterized mode.
-	event_poff_invalid	New to v6.0. Indicates an invalid value of POFF when in rasterized mode.
-	event_phase_in_invalid	New to v6.0. Indicates an invalid value of PHASE_IN when in rasterized mode.

Notes:

1. N, E, F and G are all arbitrary independent integers.

Latency Changes

In version 6.0, latency and initial conditions have changed for some configurations to ensure compliance with the equation for PHASE_OUT.

The latency of DDS Compiler v6.0 is different compared to v4.0 and greater in general. The update process cannot account for this and guarantee equivalent performance.

Note: When in blocking mode (that is, TREADY handshaking is present), the latency of the core is variable, so only the minimum possible latency can be determined. When in non-blocking mode (no TREADY), the latency of the core is as shown in the latency field of the Vivado IDE and is constant.

Behavioral Changes

SINE/COSINE lookup values remain bit accurate with respect to DDS Compiler v5.0 and v4.0. However, versions of the DDS Compiler core prior to v6.0 had different latencies for the programmable and streaming interfaces (the number of cycles between applying an input and that input affecting the calculated phase), which resulted in a different output sequence depending on the configuration.

This behavior has been standardized in DDS Compiler v6.0; however, this might not match the behavior of previous versions. The generated phase is described by the following equation:

Equation 1: Generated Phase

$$PHASE_OUT_{(n)} = \sum_{i=0}^n PINC(n) + POFF(n)$$

Instructions for Minimum Change Migration

The upgrade function alone produces a configuration of v6.0 equivalent to any configuration of v5.0. To configure the DDS Compiler v6.0 to most closely mimic the behavior of v4.0 the translation is as follows:

Parameters

Most parameters remain unchanged. Uncheck output TREADY. All other new parameters default as required for legacy operation.

If the CHANNEL output was used, set the DATA Output TUSER setting to Chan_ID_Field.

Ports

Rename ports as described in [Port Changes](#).

- WE, REG_SELECT, ADDR and DATA are replaced by the CONFIG channel, where WE is equivalent to TVALID, ADDR has no equivalent, but is replaced internally by an incrementing count with `s_axis_config_tlast` denoting the last transfer of the TDM sequence, DATA(T-1:0) is replaced by `s_axis_config_tdata` (see [CONFIG Channel TDATA Structure](#)). REG_SELECT is no longer required, because both PINC and POFF can be written in a single transfer.
- PINC_IN and POFF_IN are mapped to `s_axis_phase_tdata` as described in [Input PHASE Channel TDATA Structure](#). Connect `s_axis_phase_tvalid` to logical 1.
- SINE(P-1:0) and COSINE(P-1:0) are mapped to `m_axis_data_tdata` as described in [Output DATA Channel TDATA Structure](#). RDY becomes `s_axis_data_tvalid`.

- CHANNEL(N-1:0) becomes `m_axis_data_tuser` (N-1:0).
- PHASE_OUT(W-1:0) becomes `m_axis_phase_tdata` (W-1:0) as described in [Output PHASE Channel TDATA Structure – Conventional DDS](#).

Miscellaneous Changes

The synchronous, active-Low reset pin `aresetn` must be driven Low for a minimum of two clock cycles to correctly reset the core. The reset is registered within the core, causing some delay between the assertion/deassertion of `aresetn` and the effect being seen on the interface.

Performance

To achieve equivalent performance to v4.0, Latency should be set to the original latency value plus 1. Alternatively, set Latency to be the same as Latency for v4.0, but there might be drop in performance. Resource allocation for this configuration is greater than v4.0 in flip-flop count by approximately the number of bits in any fields in `s_axis_phase_tdata`.

Functionality Changes

Version 6.0 is bit accurate with v5.0 for any configuration of v5.0. Some configurations might differ in initial output or latency compared to v5.0 to comply with [Equation 1](#). When AXI4-Stream interface considerations have been handled according to [AXI4-Stream Considerations](#), there are no other functionality changes between v4.0 and v6.0.

Simulation

Starting with DDS Compiler v6.0 (2013.3 version), behavioral simulation models have been replaced with IEEE P1735 Encrypted VHDL. The resulting model is bit and cycle accurate with the final netlist. For more information on simulation, see the *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#)).

Debugging

This appendix includes details about resources available on the AMD Support website and debugging tools.

If the IP requires a license key, the key must be verified. The AMD Vivado™ design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation
- write_bitstream (Tcl command)



IMPORTANT! IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.

Finding Help with AMD Adaptive Computing Solutions

To help in the design and debug process when using the core, the [Support web page](#) contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support. The [Community Forums](#) are also available where members can learn, participate, share, and ask questions about AMD Adaptive Computing solutions.

Documentation

This product guide is the main document associated with the core. This guide, along with documentation related to all products that aid in the design process, can be found on the [Support web page](#) or by using the AMD Adaptive Computing Documentation Navigator. Download the Documentation Navigator from the [Downloads page](#). For more information about this tool and the features available, open the online help after installation.

Answer Records

Answer Records include information about commonly encountered problems, helpful information on how to resolve these problems, and any known issues with an AMD Adaptive Computing product. Answer Records are created and maintained daily to ensure that users have access to the most accurate information available.

Answer Records for this core can be located by using the Search Support box on the main [Support web page](#). To maximize your search results, use keywords such as:

- Product name
- Tool message(s)
- Summary of the issue encountered

A filter search is available after results are returned to further target the results.

Master Answer Record for the Core

AR [54498](#).

Sub-Harmonic Frequencies

The equations for SFDR rely on the assumption that rounding errors from the finite precision of phase and amplitude are incoherent. This assumption is violated for values of phase increment that are not mutually prime with the weight of the phase accumulator. The anomalies, such as spurs, are more obvious for larger common factors between the phase increment value and the weight of the accumulator ($2^{\text{Phase Width}}$). This is because such values might not access every location in the SIN/COS LUT, so the rounding errors are not randomly spread. To avoid this, do not use values of output frequency that are simple rational fractions of the frequency per channel, F_s , such as $3/8$ and $1/64$.

Technical Support

AMD Adaptive Computing provides technical support on the [Community Forums](#) for this AMD LogiCORE™ IP product when used as described in the product documentation. AMD Adaptive Computing cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To ask questions, navigate to the [Community Forums](#).

Debug Tools

There are many tools available to address DDS Compiler design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The AMD Vivado™ Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in AMD devices.

The Vivado logic analyzer is used to interact with the logic debug LogiCORE IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#)).

Reference Boards

Various AMD development boards support the DDS Compiler core. These boards can be used to prototype designs and establish that the core can communicate with the system.

- 7 series FPGA evaluation boards:
 - KC705
 - KC724

Interface Debug

AXI4-Stream Interfaces

If data is not being transmitted or received, check the following conditions:

- If transmit `<interface_name>_tready` is stuck Low following the `<interface_name>_tvalid` input being asserted, the core cannot send data.
- If the receive `<interface_name>_tvalid` is stuck Low, the core is not receiving data.
- Check that the `ACLK` inputs are connected and toggling.

- Check that the AXI4-Stream waveforms are being followed, as shown in [Figure 17](#), [Figure 18](#), [Figure 19](#), and [Figure 20](#).
- Check core configuration.

Additional Resources and Legal Notices

Finding Additional Documentation

Technical Information Portal

The AMD Technical Information Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Technical Information Portal, go to <https://docs.amd.com>.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the AMD Vivado™ IDE, select **Help** → **Documentation and Tutorials**.
- On Windows, click the **Start** button and select **Xilinx Design Tools** → **DocNav**.
- At the Linux command prompt, enter `docnav`.

Note: For more information on DocNav, refer to the *Documentation Navigator User Guide* ([UG968](#)).

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.
- Go to the [Design Hubs](#) web page.

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see [Support](#).

References

These documents provide supplemental material useful with this guide:

1. *Vivado Design Suite: AXI Reference Guide* ([UG1037](#))
 2. *AMBA AXI4-Stream Protocol Specification* ([ARM IHI 0051A](#))
 3. *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
 4. *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
 5. *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
 6. *System Generator for DSP User Guide* ([UG640](#))
 7. *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))
 8. *ISE to Vivado Design Suite Migration Guide* ([UG911](#))
 9. *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
 10. *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#))
-

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
05/30/2024 Version 6.0	
User Parameters	Added new SIN/COS Output Type parameter.
Configuration Tab	
Parameter Changes	
AXI Channel Options	Added details about how the output form is indirectly controlled by the SIN/COS Output Type parameter.
Hardware Parameters	Added a note about output width when selecting the floating point output type.
Implementation Tab	Added a note under Amplitude Mode about floating point selection.
System Parameters	Added a clarification about noise shaping regions.

Section	Revision Summary
01/21/2021 Version 6.0	
N/A	Added AMD Versal™ device support.
12/20/2017 Version 6.0	
Performance	Added important note.
11/18/2015 Version 6.0	
N/A	Added support for AMD UltraScale+™ devices.
06/24/2015 Version 6.0	
Chapter 6: C Model	Added MATLAB® interface description.
04/01/2015 Version 6.0	
Chapter 6: C Model	Data Format section updated to clarify sine, cosine, and phase output format.
10/01/2014 Version 6.0	
Chapter 4: Designing with the Core	Minor corrections to Phase Increment section of Designing with the Core to account for automatic gain control.
Chapter 5: Design Flow Steps	Minor corrections to Output section of Implementation Tab description in Design Flow Steps to account for Taylor mode.
Chapter 6: C Model	Added Data Format section.
04/02/2014 Version 6.0	
Resource Use	Added link to resource utilization figures.
User Parameters	Added table.
12/18/2013 Version 6.0	
N/A	Added support for UltraScale architecture.
10/02/2013 Version 6.0	
IP Facts	Minor updates.
Chapter 4: Designing with the Core	Corrections added.
N/A	Document version number advanced to match the core version number.
03/20/2013 Version 1.0	
Initial release.	Replaces DS794, LogiCORE IP DDS Compiler Data Sheet. Updated for core v6.0. Added support for Zynq 7000 and Artix 7 devices. Removed support for Design Suite. Removed support for Virtex 6 and AMD Spartan™ 6 devices.

Please Read: Important Legal Notices

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS

flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes. THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2013–2024 Advanced Micro Devices, Inc. AMD, the AMD Arrow logo, Artix, Kintex, Spartan, UltraScale, UltraScale+, Versal, Virtex, Vivado, Zynq, and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.